

Java Study

前端

JavaSE

SSM

数据库

多线程

缓存

分布式相关

调优

服务器 && Linux

前端

1. js中的=和==有什么区别, null,undefined,NaN 有什么区别

在JavaScript中，`=` 和 `==` 有非常不同的意义，而 `null`、`undefined` 和 `NaN` 都是特殊的值，它们各自有不同的含义和用途。

1. `=` 和 `==` 的区别

`=`: 这是赋值操作符。它用于将一个值赋给一个变量。例如：

```
javascript
```

```
let a = 5; // 将数字5赋给变量a
```

`==`: 这是等于操作符，用于比较两个值是否相等。注意这是宽松相等性检查，它会进行类型转换以尝试匹配值。例如：

```
javascript
```

```
let a = 5;
```

```
let b = "5";
```

```
console.log(a == b); // 输出 true, 因为JavaScript尝试将字符串"5"转换为数字5
```

如果你想进行严格的相等性检查（即不进行类型转换），你应该使用 `===` 操作符。

2. `null`、`undefined` 和 `NaN` 的区别

`null`: 表示一个空值或“无”的值。它常用于表示一个变量被有意地设置为没有值。`null` 是一个对象，但它的值是“空”或“无”。

```
javascript
```

```
let x = null; // x 是一个被赋值为 null 的变量
```

`undefined`: 表示一个变量已被声明，但还没有被赋值。访问一个未声明的变量会抛出一个错误，但访问一个未赋值的变量会返回 `undefined`。

```
javascript
```

```
let y; // y 被声明了，但还没有被赋值，所以它的值是 undefined
```

```
console.log(z); // 抛出一个错误，因为 z 没有被声明
```

`NaN`: 表示“非数字”（Not a Number）。它是当某个运算的结果不是一个数字时返回的特殊值。例如，0除以0会得到 `NaN`。

```
javascript
```

```
let result = 0 / 0; // result 的值是 NaN
```

需要注意的是，`NaN` 有一些特殊的性质。例如，`NaN` 不等于任何值，包括它自身。

```
javascript
console.log(NaN == NaN); // 输出 false
```

为了检查一个值是否是 NaN，你应该使用 isNaN() 函数或 Number.isNaN() 方法。

总结：= 和 == 是操作符，用于赋值和比较；而 null、undefined 和 NaN 是特殊的JavaScript值，它们各自表示不同的概念。

2. 什么是跨域，跨域的流程是什么，如何解决跨域

跨域：浏览器对于javascript的同源策略的限制，为了保护用户信息安全，防止恶意的网站窃取数据。

假如A网站是一家银行，用户登录之后，在用户的机器上设置了一个 Cookie，包含一些隐私信息，如存款总额。用户在离开 A 网站后，又去访问 B 网站，如果不加现在，B 网站也可以读取 A 网站的 Cookie，就会泄露隐私信息。再加上 Cookie 往往用来保存用户的登录状态，如果用户没有退出登录，其他网站就可以冒充用户，为所欲为。

解决方案：

- Jsonp：Js跨域请求数据是不可以的，但是js跨域请求js脚本是可以的。可以把数据封装成一个js语句，做一个方法的调用。跨域请求js脚本可以得到此脚本。得到js脚本之后会立即执行。可以把数据做为参数传递到方法中。就可以获得数据。从而解决跨域问题。
- nginx：服务器反向代理
- CORS：预检，在满足条件的响应头添加信息

以下情况都属于跨域：

跨域原因说明	示例
域名不同	www.jd.com 与 www.taobao.com
域名相同，端口不同	www.jd.com:8080 与 www.jd.com:8081

nginx反向配置例子：

前端server的域名为：fe.server.com

后端服务的域名为：dev.server.com

现在我在fe.server.com对dev.server.com发起请求一定会出现跨域。

现在我们只需要启动一个nginx服务器，将server_name设置为fe.server.com,然后设置相应的location以拦截前端需要跨域的请求，最后将请求代理回dev.server.com。如下面的配置：

```
server {  
    listen      80;  
    server_name fe.server.com;  
    location / {  
        proxy_pass dev.server.com;  
    }  
}
```

这样就可以完美绕过浏览器的同源策略了。

fe.server.com访问nginx的fe.server.com属于同源访问，而nginx对服务端转发的请求不会触发浏览器的同源策略。

https://blog.csdn.net/qq_46548855/article/details/129268826

https://blog.csdn.net/qq_44741577/article/details/136541798

JavaSE

1. JDK

1.1. jdk中用到了哪些设计模式

1. 单例模式 (Singleton Pattern) :

- 保证一个类仅有一个实例，并提供一个全局访问点。例如，`java.lang.Runtime`类就是单例模式的一个应用。每个Java应用程序都有一个唯一的Runtime实例，用于与运行环境进行交互¹²。

2. 工厂模式 (Factory Pattern) :

- 提供创建对象的接口，让子类决定实例化哪一个类。JDK中有很多工厂模式的例子，如`java.text.NumberFormat`的`getInstance()`方法，用于获取特定类型的数字格式化器¹²。另外，`java.util.Calendar`的`getInstance()`方法也是工厂模式的一个应用³。

3. 适配器模式 (Adapter Pattern) :

- 将一个类的接口转换成客户端所期望的另一种接口。`java.util.Arrays`的`asList()`方法就是一个适配器模式的例子，它可以将一个数组适配为一个列表 (List)¹²⁴。另外，`InputStreamReader`和`OutputStreamWriter`也是适配器模式的典型应用，它们将字节流适配为字符流⁴。

4. 观察者模式 (Observer Pattern) :

- 定义对象之间的一对多依赖关系，当一个对象改变状态时，它的所有依赖者都会收到通知并自动更新。在Java的Swing UI框架中，`java.util.Observable`类和`java.util.Observer`接口就是观察者模式的实现²。

5. 原型模式 (Prototype Pattern) :

- 用于创建重复的对象，同时又能保证性能。`java.lang.Object`的`clone()`方法就是原型模式的一个例子，它可以根据现有实例创建一个浅拷贝对象¹²。

6. 装饰器模式 (Decorator Pattern) :

- 动态地给一个对象添加一些额外的职责。Java I/O库中的类如`InputStream`、`OutputStream`、`Reader`、`Writer`及其子类就是装饰器模式的典型应用，通过包装流或读取器/写入器来添加额外的功能，如缓冲、转换等²⁴。

7. 建造者模式 (Builder Pattern) :

- 将一个复杂对象的构建与其表示分离，使得同样的构建过程可以创建不同的表示。`java.lang.StringBuilder`类就是建造者模式的一个应用，它允许我们通过一个可变的字符序列来构建字符串¹。

除了上述设计模式外，JDK中还使用了其他设计模式，如桥接模式（在AWT和JDBC中）、组合模式（在`javax.swing.JComponent`等中）等

2. ArrayList

在达到数组定义容量的时候，会自动扩容为原来的1.5倍数，默认开始容量是10；

删除和新增的时候，因为可能会涉及到多个值的移动，所以效率低于LinkedList；

但因为ArrayList是基于数组的，查询效率高于LinkedList，后者必须从头开始遍历，因为LinkedList是在内存中不连续存储的；

3. HashMap

在达到当前容量的0.75的时候，会自动扩容为原来的2倍（需要重新hash），默认开始容量是16（为了hash分布得比较均衡）；

hashmap 由数组加链表，或者数组加红黑树来存储数据；

hash 值相同的会放到同个index 下的链表，1.7之前是头插法（在链头插入，但是在并发下可能会出现循环链表），1.8及之后改成了尾插法；

在数组长度大于64，链表长度大于8的情况下，会把链表转换成红黑树；

4. 重载与重写

重载，同一个类中多个同名方法根据不同的入参来执行不同的逻辑处理。

- 在一个类里面，方法名称相同，入参的个数，类型，顺序不同；
- 方法返回值和访问修饰符可以不同。

重写，当子类继承父类的相同方法，输入数据一样，做出有别于父类的响应。override。

- 常见于父类和子类，方法名称相同，入参的个数，类型，顺序必须相同。
- 返回类型是引用类型的话，则可以为其子类；返回类型是基本数据类型和void的话，重写的时候不能变；
- 抛出的异常返回小于等于父类；
- 方法的访问修饰符大于等于父类。
- 父类方法的访问修饰符为 private、final、static 则子类不能重写该方法。
- 构造方法无法被重写，可以被重载。

5. java中的泛型及其使用场景

Java中的泛型（Generics）是JDK 5中引入的一个新特性，它提供了编译时类型安全，允许在定义类、接口和方法时使用类型参数（type parameters）。这样，我们可以编写灵活且可重用的代码，同时保持类型安全。

泛型的主要优点：

类型安全：在编译时捕获类型不匹配的错误，而不是在运行时。

代码重用：减少代码冗余，通过泛型可以编写更通用的代码。

可读性：使代码更易于理解和维护。

泛型的使用场景：

集合类：Java集合框架（如ArrayList、HashSet等）大量使用了泛型。通过使用泛型，我们可以创建特定类型的集合，例如ArrayList<String>或HashSet<Integer>，从而在编译时确保集合中只包含特定类型的元素。

```
List<String> stringList = new ArrayList<>();
```

```
stringList.add("Hello"); // 正确
```

```
// stringList.add(123); // 编译错误，因为集合只接受String类型
```

自定义类和方法：我们也可以在自已的类和方法中使用泛型。例如，我们可以创建一个泛型类来表示一对元素，或者创建一个泛型方法来交换两个元素的值。

```
public class Pair<T> {
```

```
    private T first;
```

```
    private T second;
```

```
    // 构造器、getter和setter等
```

```
}
```

```
public static <T> void swap(T[] array, int i, int j) {
```

```
    T temp = array[i];
```

```
    array[i] = array[j];
```

```
array[j] = temp;
}
```

泛型接口：泛型也可以用于接口。这允许我们创建更通用的接口，其实现可以处理不同的类型。

```
public interface GenericInterface<T> {
    void performOperation(T value);
}
```

通配符类型：Java泛型还提供了通配符类型（?），用于表示未知类型。这在处理不同类型的集合时非常有用。例如，我们可以使用List<?>来表示一个未知类型的列表。此外，还有上界通配符（? extends Type）和下界通配符（? super Type），用于表示类型参数的上界和下界。

```
public void printList(List<?> list) {
    for (Object elem : list) {
        System.out.println(elem);
    }
}
```

类型擦除：值得注意的是，Java的泛型是通过类型擦除实现的。这意味着在运行时，泛型类型信息会被擦除，所有的泛型类型都将被替换为其限定类型（如果有的话，否则为Object）。因此，我们不能在运行时检查泛型类型，也不能创建泛型类型的Class实例。

6. java中的lambda表达式及其用途

在Java中，Lambda表达式是一种简洁、可读的函数式编程构造，用于表示匿名函数。它允许你以简洁的方式表示实例化的函数式接口。Lambda表达式的主要用途是简化代码，提高代码的可读性和可维护性，特别是在处理集合、并发编程以及编写流式处理逻辑时。

Lambda表达式的语法如下：

`(parameters) -> expression`

或

`(parameters) -> { statements; }`

`parameters` 是Lambda表达式的参数列表，它与函数式接口中抽象方法的参数列表相匹配。

`->` 是Lambda操作符，它分隔参数列表和Lambda体。

`expression` 或 `{ statements; }` 是Lambda体，它定义了Lambda表达式的行为。如果Lambda体只包含一个表达式，则可以使用简化语法。如果Lambda体包含多条语句，则必须使用花括号将它们括起来。

Lambda表达式的一个关键前提是它们必须与函数式接口一起使用。函数式接口是只有一个抽象方法的接口，Lambda表达式可以被视为该抽象方法的实现。从Java 8开始，`@FunctionalInterface`注解可以用来标记一个接口为函数式接口，但即使没有这个注解，只要接口确实只定义了一个抽象方法，它也可以被用作Lambda表达式的目标类型。

Lambda表达式的用途非常广泛，以下是一些具体的应用场景：

集合操作：在Java的集合框架中，Lambda表达式经常用于简化迭代和过滤操作。例如，使用`forEach`、`map`、`filter`等Stream API方法时，Lambda表达式可以方便地定义操作逻辑。

```
List<String> list = Arrays.asList("apple", "banana", "cherry");
```

```
list.forEach(item -> System.out.println(item));
```

事件处理：在图形用户界面（GUI）编程中，Lambda表达式经常用于事件监听器，比如按钮点击事件的处理。

```
button.addActionListener(e -> System.out.println("Button clicked!"));
```

线程与并发：在Java的并发API中，Lambda表达式常用于简化线程和任务的创建。例如，使用ExecutorService时，你可以直接传递一个Lambda表达式作为Runnable或Callable的实现。

```
ExecutorService executor = Executors.newSingleThreadExecutor();

executor.submit(() -> {
    // Task logic here

    System.out.println("Task executed by thread: " + Thread.currentThread().getName());
});

executor.shutdown();
```

Comparator与排序：Lambda表达式可以方便地定义比较逻辑，用于集合的排序操作。

```
List<String> list = Arrays.asList("apple", "cherry", "banana");

Collections.sort(list, (s1, s2) -> s1.compareTo(s2));
```

简化匿名内部类：在Java 8之前，我们经常使用匿名内部类来实现接口。Lambda表达式提供了一种更简洁、更可读的方式来达到同样的目的。

SSM

spring

1. 常用注解

1. **@Autowired**: 用于自动装配bean, 可以放在类成员变量上, 或者放在setter方法、构造器上。当Spring遇到这个注解时, 它会在上下文中查找合适的bean来自动装配。
2. **@Qualifier**: 与@Autowired一起使用时, 可以帮助指定要注入bean的名称, 当存在多个相同类型的bean时特别有用。
3. **@Component**: 表明一个类会作为Spring组件, 让Spring IoC容器自动检测并注册到容器中。它实际上是一个泛化的概念, 可以被@Service、@Repository、@Controller等注解替代。
4. **@Service**: 用于标注业务逻辑组件, 通常用于服务层。
5. **@Repository**: 用于标注数据访问组件, 即DAO组件。
6. **@Controller**: 用于标注控制器组件, 通常与Spring MVC一起使用。
7. **@RestController**: 是@Controller和@ResponseBody的组合注解, 用于创建RESTful Web服务。
8. **@RequestMapping**: 用于映射web请求(如URL路径)到控制器中的特定处理方法。
9. **@GetMapping** 、 **@PostMapping** 、 **@PutMapping** 、 **@DeleteMapping** 、 **@PatchMapping**: 是@RequestMapping的简化版, 分别对应不同的HTTP请求方法。
10. **@PathVariable**: 用于从URI模板变量中获取值, 并绑定到控制器方法的参数上。
11. **@RequestParam**: 用于绑定请求参数到控制器方法的参数。
12. **@ResponseBody**: 将控制器方法返回的对象转换为JSON或XML格式的响应体。
13. **@ModelAttribute**: 用于添加模型属性, 通常用于数据绑定。
14. **@Value**: 用于注入外部配置文件的属性值到bean的属性中。
15. **@Configuration**: 声明当前类是一个配置类, 用于定义bean。
16. **@Bean**: 在配置类中通过该方法返回一个对象, 这个对象会注册为Spring应用上下文中的bean。
17. **@Profile**: 用来定义特定环境下生效的bean定义或配置类。
18. **@Scope**: 用于定义bean的作用域, 如singleton(单例)、prototype(原型)等。
19. **@Async**: 用于声明异步方法, 使得被标注的方法可以异步执行。
20. **@EnableScheduling** 和 **@Scheduled**: 用于启用定时任务功能, @Scheduled用于标注需要定时执行的方法。
21. **@ComponentScan**: 配置组件扫描的路径, 让Spring可以扫描到带有@Component等注解的类。
22. **@EnableWebMvc**: 开启Spring MVC的支持。

- 23. **@EnableTransactionManagement**: 开启声明式事务管理。
- 24. **@Transactional**: 用于声明事务边界，标注在类或方法上，表示被标注的方法或类中的方法运行在一个事务上下文中。
- 25. **@PropertySource**: 加载外部配置文件。
- 26. **@Import**: 用于导入其他配置类。

2. spring中用到的设计模式

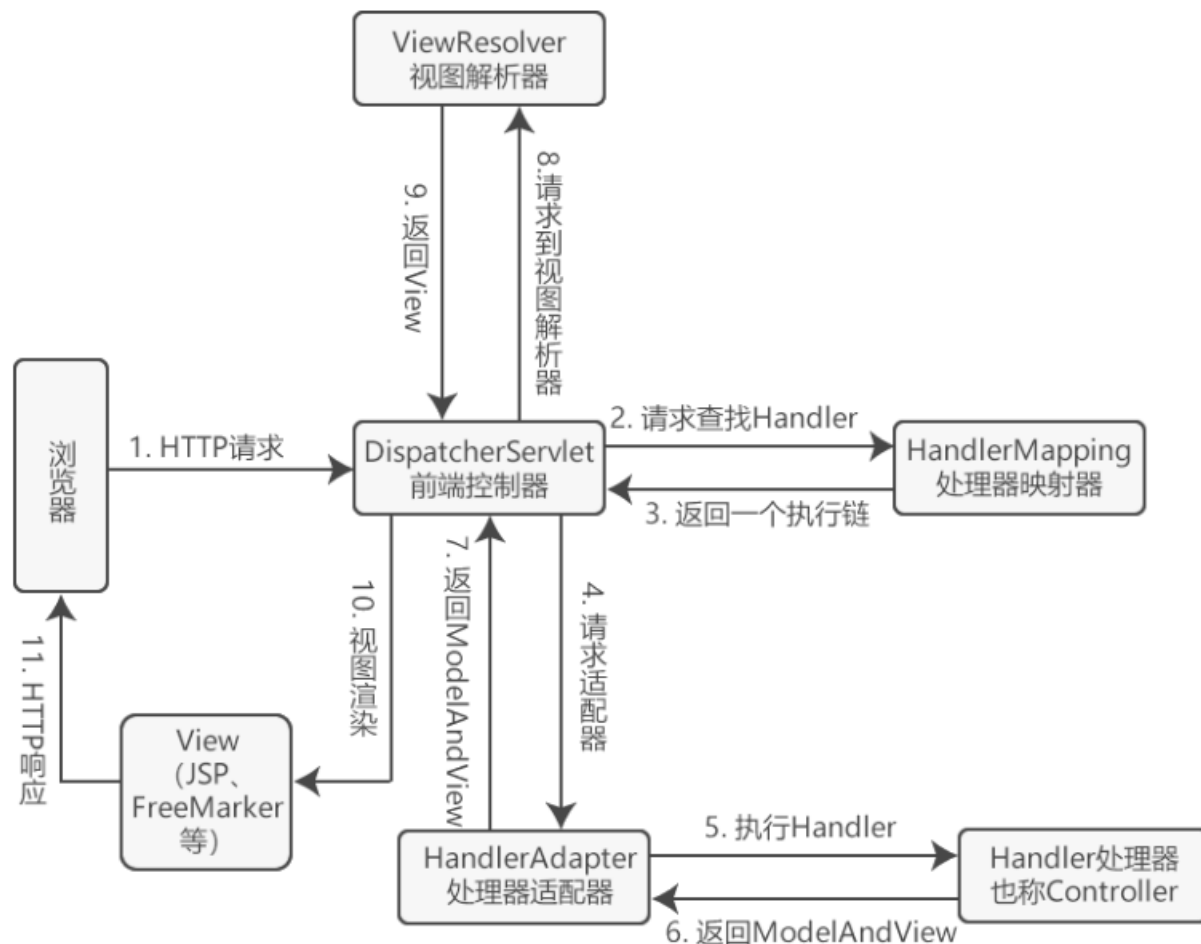
- 工厂模式：Spring使用工厂模式，通过BeanFactory和ApplicationContext来创建对象；
- 单例模式：Bean默认为单例模式；
- 策略模式：例如Resource的实现类，针对不同的资源文件，实现了不同方式的资源获取策略；
- 代理模式：Spring的AOP功能用到了JDK的动态代理和CGLIB字节码生成技术；
- 模板方法：可以将相同部分的代码放在父类中，而将不同的代码放入不同的子类中，用来解决代码重复的问题。比如RestTemplate, JmsTemplate, JpaTemplate；
- 适配器模式：Spring AOP的增强或通知（Advice）使用到了适配器模式，Spring MVC中也是用到了适配器模式适配Controller；
- 观察者模式：Spring事件驱动模型就是观察者模式的一个经典应用；
- 桥接模式：可以根据客户的需求能够动态切换不同的数据源。比如我们的项目需要连接多个数据库，客户在每次访问中根据需要会去访问不同的数据库；

<https://blog.csdn.net/a745233700/article/details/112598471>

SpringMVC

1. 工作原理

1. 用户点击界面，浏览器发起一个HTTP请求，该请求被提交到 DispatcherServlet （前端处理器）；
2. DispatcherServlet 请求一个或多个 HandlerMapping （处理器映射器），返回一个执行链（HandlerExecutionChain）；
3. DispatcherServlet 将执行链返回的 Handler 信息发给 HandlerAdapter （处理器适配器）；
4. HandlerAdapter 根据 Handler 信息找到并执行响应的 Handler (也称为 Controller)；
5. Handler执行后，返回结果 ModelAndView 给到 HandlerAdapter ；
6. HandlerAdapter 返回结果给 DispatcherServlet；
7. DispatcherServlet 其请求 ViewResolver （视图解析器）解析；
8. ViewResolver 根据 View 信息匹配到相应视图结果，返回给 DispatcherServlet；
9. DispatcherServlet 接收到视图后，进行渲染，将 Model 中的模型数据填充到 View 视图中，生成最终视图
10. 视图返回给浏览器响应



2. RestController 和 Controller 注解

`@RestController = @Controller + @ResponseBody`

`RestController` 注解，把方法的返回结果序列化为 JSON 或 XML 返回给客户端，常用于 restful 接口。

`Controller` 注解，方法的返回结果默认是视图名称，JSP页面所对应的名称。

3. 常用注解

1. **@Controller**: 用于标记在一个类上, 表示该类是一个SpringMVC Controller对象。分发处理器将会扫描使用了该注解的类的方法, 并检测该方法是否使用了@RequestMapping注解¹²。
2. **@RequestMapping**: 用于配置映射路径, 即将HTTP请求的URL映射到一个具体的处理方法。它可以用于类或方法上, 当用于类上时, 表示类中的所有响应请求的方法都以该地址作为父路径¹²³。
3. **@RequestParam**: 用于提取请求参数的值³⁴。
4. **@PathVariable**: 用于绑定URL模板变量值到控制器方法的参数上³⁴。
5. **@GetMapping** 、 ****@PostMapping** 、 **@PutMapping** 、 **@DeleteMapping** 、 **@PatchMapping********: 这些注解都是对HTTP请求方法的简化表示, 用于处理特定类型的请求, 如GET、POST、PUT、DELETE、PATCH等³。
6. **@ResponseBody**: 将返回的对象序列化为JSON或XML响应体返回给客户端³。
7. **@RequestBody**: 用于将请求体中的JSON或XML数据绑定到方法的参数上³⁴。
8. **@Service**: 用于在服务层(Service)标识一个类为Spring的服务组件⁴。
9. **@Repository**: 用于在数据访问层(Repository)标识一个类为Spring的数据访问组件⁴。
10. **@Entity**: 在实体类(Entity)中标识一个类为JPA实体⁴。
11. **@Autowired**: 用于自动装配bean, 它可以对类成员变量、方法及构造函数进行标注, 完成自动装配的工作³。
12. **@Resource**: 用于bean的注入时使用, 可以写在字段和setter方法上。虽然@Resource并不是Spring的注解, 但Spring支持该注解的注入²。
13. **@ModelAttribute**: 用于将请求参数绑定到模型对象, 常用于数据绑定³。
14. **@SessionAttributes**: 用于将值放到session域中³。
15. **@RestController**: 这是一个特殊的控制器注解, 其内部默认包含了@Controller和@ResponseBody, 用于创建RESTful Web服务¹⁴。
16. **@ControllerAdvice**、****@RestControllerAdvice****: 用于全局异常处理、全局数据绑定等¹。
17. **@ComponentScan**: 用于配置Spring扫描组件的路径⁴。
18. **@Configuration**: 用于定义配置类, 声明一个或多个@Bean方法⁴。
19. **@Bean**: 用于在配置类中定义创建并返回一个对象的方法, 这些对象将被注册为Spring应用上下文中的bean³。

MyBatis

1. MyBatis是怎么进行分页的

MyBatis 提供了分页查询的功能，但需要注意的是，MyBatis 本身并不直接提供分页插件，而是通常通过一些第三方插件，如 PageHelper，或者自定义 SQL 来实现分页。

以下是两种常见的分页实现方式：

1. 使用第三方插件，如 PageHelper

PageHelper 是一个非常好用的 MyBatis 分页插件，它可以帮你自动完成分页的 SQL 查询，而无需你手动编写分页的 SQL 语句。

使用步骤：

首先，你需要将 PageHelper 插件添加到你的项目中。

在 MyBatis 的配置文件中配置 PageHelper 插件。

在你的查询方法中，使用 PageHelper 的静态方法进行分页设置。

执行查询，获取分页结果。

示例代码：

// 在你的查询方法中

```
PageHelper.startPage(pageNum, pageSize); // pageNum 为当前页码，pageSize 为每页显示的记录数
```

```
List userList = userMapper.selectUser(); // 执行查询，获取分页结果
```

2. 自定义 SQL 实现分页

你也可以通过自定义 SQL 语句来实现分页查询。这种方式需要你对 SQL 有一定的了解，因为你需要手动编写分页的 SQL 语句。

对于 MySQL 数据库，你可以使用 LIMIT 和 OFFSET 关键字来实现分页。

示例 SQL 语句：SELECT * FROM user LIMIT 10 OFFSET 20;

这条 SQL 语句会查询 user 表中的第 21 到 30 条记录（因为 OFFSET 是从 0 开始的）。

在 MyBatis 的映射文件中，可以将这条 SQL 语句作为查询语句，然后通过参数传递 LIMIT 和 OFFSET 的值来实现动态分页。

需要注意的是，不同的数据库有不同的分页语法，你需要根据你所使用的数据库来编写相应的分页 SQL 语句。

2. 分页插件 PageHelper 是怎么工作的

PageHelper进行分页的原理主要基于MyBatis的插件机制。它是一个开源的MyBatis分页插件，通过实现一个PageInterceptor拦截器并加载到MyBatis的拦截器链中，达到自动分页查询的效果。

在使用时，首先通过PageHelper.startPage这样的语句在当前线程上下文中设置一个ThreadLocal变量，然后当MyBatis执行查询时，PageInterceptor拦截器会拦截这个查询，从ThreadLocal中拿到分页的信息。如果检测到分页信息，拦截器会自动在原始的SQL语句后面拼装分页的SQL（如limit语句等），从而实现分页查询。最后，分页查询完成后，ThreadLocal中的分页信息会被清除。

通过这种方式，PageHelper大大简化了分页查询的复杂性，减少了开发人员编写复杂的分页SQL语句的工作量，同时还支持多种数据库和多种分页方式。并且，PageHelper还提供了排序和筛选的功能，支持插件扩展，可以根据需求进行自定义。

注意，PageHelper使用ThreadLocal保存分页参数，因此分页参数与线程是绑定的，这意味着每次分页查询都需要在独立的线程环境中进行，避免多线程环境下的分页参数冲突问题。

总的来说，PageHelper是一个强大且方便的工具，可以大大简化在MyBatis中进行分页查询的操作。但在使用时，也需要注意其使用限制和潜在问题。

数据库

1. 分库分表：

1. 分库：

- 垂直分库，根据业务和用途，把多个表分成多个数据库
- 水平分库，在主从架构的情况下，写请求过多，一个master负载不了的情况下，可以把一个表复制多份放到多个数据库中，减轻压力。

2. 分表：

- 水平分表，因为数据量过大，可以把一个表分成同样表结构的多个表
- 垂直分表，因为业务需求增长，表字段过多，可以把一个表拆分成多个表，访问频繁的字段放到一个表，如订单表与订单详情表，用户信息表与用户扩展信息表等

3. 分库分表：

- 可以把数据按年份分库，按月份分表，方便统计和查询某一段时间内的数据

2. oracle 分区

https://blog.csdn.net/qq_32445015/article/details/132624722

3. 事务特性

- 原子性 (Atomicity) , 可以理解为一个事务内的所有操作要么都执行, 要么都不执行。
- 一致性 (Consistency) , 可以理解为数据是满足完整性约束的, 也就是不会存在中间状态的数据, 比如你账上有400, 我账上有100, 你给我打200块, 此时你账上的钱应该是200, 我账上的钱应该是300, 不会存在我账上钱加了, 你账上钱没扣的中间状态。
- 隔离性 (Isolation) , 指的是多个事务并发执行的时候不会互相干扰, 即一个事务内部的数据对于其他事务来说是隔离的。
- 持久性 (Durability) , 指的是一个事务完成了之后数据就被永远保存下来, 之后的其他操作或故障都不会对事务的结果产生影响。

4. 事务隔离级别

1. SQL标准定义的事务隔离级别：

- 读未提交 (READ UNCOMMITTED) ：一个事务还没提交时，它做的变更就能被别的事务看到。
- 读提交 (READ COMMITTED) ：一个事务提交之后，它做的变更才会被其他事务看到。
- 可重复读 (REPEATABLE READ) ：一个事务执行过程中看到的数据，总是跟这个事务在启动时看到的数据是一致的。当然在可重复读隔离级别下，未提交变更对其他事务也是不可见的。
- 串行化 (SERIALIZABLE) ：对于同一行记录，“写”会加“写锁”，“读”会加“读锁”，当出现读写锁冲突的时候，后访问的事务必须等前一个事务执行完成，才能继续执行。

2. 隔离级别解决了的问题分别有：

- 脏读 (dirty read) ：一个事务读到了另一个未提交事务修改过的数据
- 不可重复读 (non-repeatable read) ：如果一个事务只能读到另一个已经提交的事务修改过的数据，并且其他事务每对该数据进行一次修改并提交后，该事务都能查询得到最新值。
- 幻读 (phantom read) ：如果一个事务先根据某些条件查询出一些记录，之后另一个事务又向表中插入了符合这些条件的记录，原先的事务再次按照该条件查询时，能把另一个事务插入的记录也读出来。

3. MVCC (Multi-Version Concurrency Control , 多版本并发控制) 指的就是在使用 读已提交 (READ COMMITTED) 、可重复读 (REPEATABLE READ) 这两种隔离级别的事务在执行普通的SELECT操作时访问记录的版本链的过程，这样子可以使不同事务的读-写、写-读操作并发执行，从而提升系统性能。

4. oracle 只支持读提交和串行化的事务隔离级别；MySQL支持以上四种事务隔离级别。

5. mysql的默认事务隔离级别是可重复读，oracle的默认事务隔离级别是读已提交。

5. Oracle不支持读未提交(READ UNCOMMITTED)级别，因为Oracle的多版本一致性特性天然地提供了非阻塞读，从而避免了脏读的可能性

隔离级别	脏读 (Dirty Read)	不可重复读 (NonRepeatable Read)	幻读 (Phantom Read)
未提交读 (Read uncommitted)	可能	可能	可能
已提交读 (Read committed) I	不可能	可能	可能
可重复读 (Repeatable /rɪ'pitəbl/ read)	不可能	不可能	可能
可串行化 (Serializable)	不可能	不可能	不可能

CSDN @Java超神之路

5. 索引

5.1. 索引选型:

hash: 通过hash算法映射, 类似于数组下标可以快速找到数组值, 适用于查找精确的值, 不适用于范围查找。适用于K-V类型的数据库, 如redis

二叉树: 有序, 二分查找可以查询精确的值, 也使用与范围查询。

查询和更新的时间复杂度都保持 $O(\log(N))$ 的话, 需要是完全平衡二叉树。

但索引也需要持久化到磁盘, 数据多的话, 树会很高, 增加磁盘IO成本, 所以需要B树, B数一个节点可以存放多个元素, 提高磁盘IO的效率。

而B+树会把非叶子结点的元素冗余一份放到叶子结点中, 同时存放指向下一个节点的指针, 能够提高范围查找的效率。

5.2. Mysql 和 Oracle 的索引区别:

mysql的Innodb引擎，是用的B+树作为索引。

oracle，是用的B树作为索引。

B树更适合需要频繁插入，删除和修改操作的数据库，B+树则在查询效率和数据存储方面更有优势。

5.3. Mysql选用唯一索引还是普通索引：

change buffer。

当需要更新一个数据页的时候，如果数据页已经在内存中则直接更新，否则在不影响数据一致性的前提下，InnoDB会将这些更新操作缓存在change buffer中，这样就不需要读取这个数据页了。

在下次查询需要访问这个数据页的时候，将数据页加载进内存，然后执行change buffer中与这个数据页有关的操作，保证这个数据逻辑的正确性。

change buffer，实际上是可以持久化的数据，也会被写入到磁盘中。将change buffer中的操作应用到原数据页，得到最新结果的过程称为merge。

除了访问这个数据页会触发merge外，系统有后台线程会定期merge。在数据库正常关闭的过程中，也会执行。

通过将更新操作先记录再 change buffer，减少读磁盘，语句的执行速度会得到明显的提升。而且，数据读入内存是需要占用buffer pool的，所以这种还能避免占用内存，提高内存利用率。

对于唯一索引来说，所有更新操作都需要先判断下这个操作是否违反唯一性约束，必须把数据加载到内存中，直接更新就行，没必要把更新操作缓存起来了，所以唯一缓存不用change buffer。

普通索引可以用change buffer。所以一般来说，一般用普通索引，有change buffer 这个机制减少了随机磁盘的访问，对更新性能的提升是会很明显的。

change buffer 是 buffer pool里的内存，可以通过参数 innodb_change_buffer_max_size来动态设置。

change buffer适用于写多读少的业务，把多次记录变更动作缓存下来，后面merger的时候收益会更大，节省更多时间，常见于账单类，日志类的系统。

但是更新之后，马上就访问的话，change buffer这种反而起到反面作用。

5.4. explain 执行计划与执行明细

▼ Explain: 可以让我们查看MYSQL执行一条SQL所选择的执行计划;

- id: 序号从大到小执行, 相同的话从上往下执行。
- select_type: 主要为下面这几种
 - a. SIMPLE 简单select查询, 不包含子查询和union
 - b. PRIMARY 复杂查询中的最外层查询, 表示主要的查询
 - c. SUBQUERY select或where列表中包含了子查询
 - d. DERIVED from列表中包含了子查询, 即衍生
 - e. UNION union之后的查询
 - f. UNION RESULT 从union后的表获取结果集
- table: 表名称
- partitions: 匹配的分区
- type: 连接类型
 - i. system: 表只有一条记录
 - ii. const: 通过索引一次就能找到
 - iii. eq_ref: 用于主键或唯一索引扫描
 - iv. ref: 用于非主键和唯一索引扫描
 - v. fulltext: 全文扫描
 - vi. ref_or_null: 类似于ref, 还会扫描null的数据
 - vii. index_merge: 多种索引合并的方式扫描
 - viii. unique_subquery: 类似于eq_ref, 条件用于in子查询
 - ix. index_subquery: 类似于unique_subquery, 条件用于in子查询, 但条件非主键或唯一索引
 - X. range: 范围扫描
 - Xi. index: 全索引扫描
 - Xii. ALL: 全表扫描
 - Xiii. 执行结果最好的是从上到下, 重点关注system > const > eq_ref > ref > range > index > ALL
- possible_keys: 可能的索引选择
- key: 实际用到的索引
- key_len: 实际索引长度, 由字符集, 长度和是否为空组成, 值小于索引的长度的话会索引使用不充分。
- ref: 与索引比较的列, 索引命中的列或常量。

- rows: 预计要检查的行数, 估值。
- filtered: 按表条件过滤的行百分比, 表示按表条件过滤的表行的估计百分比。
- Extra: 附加信息, 常见的有:
 - Impossible where: where后面的条件不成立
 - Using filesort: 表示按文件排序, 一般是在指定的排序和索引排序不一致的情况下才会出现
 - Using index: 表示是否用了覆盖索引, 表示所有获取的列是否都走了索引
 - Using temporary: 表示是否用了临时表, 一般多见于order by和group by 语句
 - Using where: 表示是否使用了where条件过滤
 - Using join buffer: 表示是否使用了连接缓存

<https://mp.weixin.qq.com/s/o-BVtbmgHQjwnGhvfK-Nsw>

用explain进行索引优化的过程:

- 1.先用慢查询日志定位具体需要优化的sql
- 2.使用explain执行计划查看索引使用情况
- 3.重点关注:

key (查看有没有使用索引)

key_len (查看索引使用是否充分)

type (查看索引类型)

Extra (查看附加信息: 排序、临时表、where条件为false等)

一般情况下根据这4列就能找到索引问题。

- 4.根据上1步找出的索引问题优化sql
- 5.再回到第2步

▼ Profiling: 可以用来准确定位一条SQL的性能瓶颈;

1, 开启profiling: set profiling=1;

2, 执行QUERY, 在profiling过程中所有的query都可以记录下来;

3, 查看记录的query: show profiles;

4, 选择要查看的profile: show profile cpu, block io for query 6;

status是执行SQL的详细过程;

Duration: 执行的具体时间;

CPU_user: 用户CPU时间;

CPU_system: 系统CPU时间;

Block_ops_in: IO输入次数;

Block_ops_out: IO输出次数;

profiling只对本次会话有效;

5.5. 索引失效的场景

https://mp.weixin.qq.com/s?__biz=MzkwNjMwMTgzMQ

1. 不满足最左匹配原则：即联合索引的第一个字段在where后面出现了就会走索引
2. 使用了select *
3. 索引上有计算，用了函数
4. 字段类型不同，隐形转换
5. like的左边包含了%
6. 列进行了对比，如where height = id, 尽管两个都是索引的话，也是不走索引
7. or关键字用法错误：or连接的条件必须都是索引的字段，一个不是的话就直接不走索引了
8. not in和not exists：主键字段使用not in可以走索引，但是普通索引或是其他用not in 不走索引；not exists不走索引；
9. order by：
 - 没有where或者limit限制的话，不走索引
 - order by 多个不同索引的字段也不走索引
 - order by 后面的字段不满足最左原则
 - order by 后面的字段，排序的方法不同，如一个升序，一个降序

6. 你做过什么数据库优化操作？平时写SQL需要注意什么？

也即是 mysql索引常见优化

1. 覆盖索引：查询的值都是索引字段，也就是可以把经常查询的字段作为一个联合索引。覆盖索引可以解决回表场景，即select * from a where a.name = 'XXX' 这种，会先根据索引name查询到主键id为2，再根据主键为2去查主键索引。

尽量不要select *, 尽量用覆盖索引

2. 最左匹配原则：多个列组成的联合索引，索引只能用于查询key是否相等，遇到范围查询(>, <, between, like)这种就不能进一步匹配了，后续退化为线性查找。同时，索引中列的顺序也决定了可命中索引的列数。

按照索引定义的字段顺序写sql的查询条件

3. 合并索引：尽量使用多个单列索引，然后将结果用union, and等连接合并起来。

4. 索引列不参与计算，不做函数操作。

5. 重复，值多次出现的列不适合作为索引

6. 多次查询的列可以作为索引

7. 不同字符集的表，联表查的时候可能不走索引

8. 更新非常频繁的字段不适合作为索引，有维护成本

9. 不会出现在where 子句中的字段不应该作为索引

10. 只为必要的列创建索引，索引需要单独维护，数据增删改都会改动到索引

11. 隐式类型转换也会不走索引。如 A.id = 1, id是字符型，但是用数字类型的去查找，会 CAST(id AS signed int) 转换，全表扫描

12. 对于很长的字段值，可以用前缀索引来加快查询速度。如网站 www.baidu.com,可以截取中间部分建索引，身份证号码可以反转之后再建索引

13. 小表驱动大表。如in的左边一般是大表，右边是小表，效率会好点；left join 的时候，左表尽量是小表；

14. 可以用limit来减少下影响或查询的记录数量

15. 可以从上页的最大id开始来优化分页，如limit 10000, 20 变成 id > 10000 limit 20

16. 可以用连接查询（内连接）来代替子查询，如 select id, name from user where userid in (select userid from order where status = 1) 变成 select id, name from user inner join order on user.userid = order.userid and order.status = 1

17. join表优化：

- 1, 尽可能减少Join 语句中的Nested Loop 的循环总次数，用小结果集驱动大结果集；
- 2, 优先优化Nested Loop 的内层循环；
- 3, 保证Join 语句中被驱动表上Join 条件字段已经被索引；
- 4, 扩大join buffer的大小；

7. 假如一个sql查询时间很长，有什么排除思路或优化思路？

即sql优化步骤，原则，方法等。

1、选择需要优化的SQL

2、Explain和Profile入手

1、任何SQL的优化，都从Explain语句开始；Explain语句能够得到数据库执行该SQL选择的执行计划；

2、首先明确需要的执行计划，再使用Explain检查；

3、使用profile明确SQL的问题和优化的结果；

3、永远用小结果集驱动大的结果集

4、在索引中完成排序

5、使用最小Columns

6、使用最有效的过滤条件

7、避免复杂的JOIN和子查询

8. 怎么处理报表，海量历史数据的导出导入？有没有异步？

导出：

1. 同步导出数据的话，很容易接口超时：异步方式解决。可以用Job或者是MQ。

- Job：增加一张任务表，记录每次导出的任务。定义一个Job，每隔一段时间，扫描一段次执行任务表，

查出所有待执行的记录，修改状态为执行中（避免没执行完，然后又到了执行时间，重复执行）然后挨个执行，完成之后，记录状态为成功或者失败。适用于对时间不敏感的业务场景。

- MQ：点击导出之后，向mq服务端发送一条消息。然后有个专门的mq消费者消费消息。对于处理失败的情况，可以增加补偿机制，自动发起重试，将消息放入死信队列。

2. 把数据一次性加载到内存，很容易引起OOM：使用阿里开源工具esayExcel，把数据逐行解析处理

3. 数据量太大，sql执行很慢：分页查询，分批处理

4. 走异步，如何通知用户导出结果：

1) 上传文件到专门的OSS服务器

2) 使用webSocket来实时通知。可以增加一张专门的通知表，记录webSocket推送的通知标题，用户，附件地址，阅读状态，类型等信息，方便追溯通知记录。webSocket给客户端推送一个通知之后，用户的有商家的收件箱，实时出现了一个小窗口，提示本次导出excel的是成功还是失败，并且有文件下载链接。当前通知的阅读状态是未读，用户点击窗口后，可以看到通知的详细内容，然后通知状态变成已读。

除了一些数据迁移等的需要，一般的业务需求应该不会涉及到导出海量数据的，违反安全审计之类的，所以这个业务本身也是一个解答的入口。

导入：

- 阿里开源工具EasyExcel，每读取10000条(自定义)，就批量插入数据库一次。
- 多线程，使用线程池

导入之后，需要检查下导入数据库的条数是否与excel中的一致，不一致的话，拿到没插进去的数据再进行分析；

假如excel中有重复的数据，可以把某个字段设置为主键，或者insert ignore来处理；

9. 设计数据库，表的时候，会注意什么？

1. 名字：见名知意，小写，_作为连接词，表名、字段名、索引名可以加业务前缀
2. 字段类型：
 - 尽量选占用存储空间下的，在满足正常业务需求情况喜爱，从小到大，往上选
 - 字符串固定或差别不大，可选择char。差别较大的可以选varchar
 - 是否字段可以选bit类型
 - 枚举字段可以选tinyint类型
 - 主键字段可以选bigint类型
 - 金额字段可以选decimal类型
 - 时间字段可以选timestamp或datetime类型
3. 字段长度
4. 字段数量：尽量不超过20个
5. 主键
6. mysql存储引擎：尽量选择innodb存储引擎，mysql8之后的默认选择。
7. not null：字段尽量设置成not null，表新增非空列的时候要设置默认值，兼容系统的原insert语句
8. 不使用外键
9. 建立索引，尽量不超过5个
10. 推荐优先使用datetime来保存日期
11. 推荐优先使用decimal来保存金额
12. 唯一索引
13. mysql建议在建表时字符集设置成：utf8mb4
14. mysql需要注意字符排序集（和字符集相关联）
15. 大字段：选用正确合适的方法。如评论用varchar(50)，不用text。合同的话可以用mongodb存储具体内容。

10. select for update加的是什么锁

主要看语句中 where 后跟的字段

主键索引，唯一索引的话，加的是行锁；有几条满足条件的就是加了几行的行锁

普通索引，非索引字段，还有查询结果是空的话，加的是间隙锁（锁的是一个范围）

在select for update之后，事务没提交的情况下，如果有别的错误想要操作锁住的行，就会一直等待，知道超时。

11. count用法

- count(*), 获取所有行数据，不做任何处理，行数加一
- count(1), 获取所有行数据，每行固定值1，行数加一
- count(id), id为主键，获取所有非空的行数据，行数加一
- count(普通索引), 获取所有非空的普通索引列，行数加一
- count(未加索引列), 获取所有非空的未加索引列，行数加一
- 性能从高到低是 count(*) 约等于 count(1) > count(id) > count(普通索引列) > count(未加索引列)

12. 怎么对加密之后的数据进行模糊查询

第一种：

- 数据量不多的话，可以把数据加载到内存中，解密之后，再用String的API进行查询

第二种：

1. 把原数据进行拆分，按照实际进行拆分，分段，然后每段加密之后存到数据库表的一个字段，每段直接用符号隔开，接着查询的之后就可以直接用 like 进行查询了。
2. 分段如：18099767651，每三个分一段为 180, 809, 099, 997, 976, 767, 676, 765, 651

13. SQL注入

通过让应用执行一些恶意SQL获取信息，达到获取数据库全部数据，删除数据库，把系统搞挂等效果。一般是因为没有用预编译语句执行的原因。

如 `select * from t_order order by orderid; select * from user; --limit 20,10;`

可以采用预编码的语句来执行SQL，在mybatis可以用#来接受参数。preparestatement预编译机制会在sql语句执行前，对其进行语法分析、编译和优化，其中参数位置使用占位符?代替。当真正运行的时候，传过来的参数会被看做是一个纯文本，字符串，不会重新编译，不会被当做sql指令。

比如上面的会变成 `select * from t_order order by 'orderid; select * from user; --' limit 20,10;`

当在特殊场景下需要使用\$来接收参数时，可以自定义util工具过滤所有注入关键字。或者用的是druid的连接池的话，可以开启防火墙中的filter中的防火墙来防止。

如何防止SQL注入：

- 使用预编译机制
- 对特殊字符转义
- 捕获异常，不暴露敏感的sql异常
- 使用sqlMap等代码检测工具
- 要有监控，有异常情况，及时邮件或短信提醒
- 数据库账号需要控制权限。对于生产环境的数据库建议单独的账号，只分配DML的相关权限，且不能访问系统表。不要在程序中直接使用管理员账号。
- 代码review, 找出部分隐藏问题。
- 使用其他手段。如自己过滤，或者是开启druid的filter防火墙。

14. varchar(64)的，直接扩容到varchar(640)的会有什么后果

1. 浪费磁盘存储空间
2. 查询性能下降：在执行查询操作的时候，数据库需要读取和处理更多的数据量，导致查询速度变慢，特别是在对大量数据进行查询时。
3. 索引效率降低：假如字段需要作为索引，索引大小受到字段大小的影响，字段越大，索引占用的磁盘空间就会增加，索引效率也会降低。
4. 降低可读性和可维护行

https://deepinout.com/mysql/mysql-questions/336_hk_1711319420.html

15. B树（B-树）和 B+树

B树：内部所有节点都可以存储数据和指针

B+树：

- 只有叶子节点存储数据，且叶子结点之间通过链表连接，适用于范围查询和搜索，查询性能也更均衡；
- 非叶子结点存储指针，一个节点内可以存更多的键，减少树的高度

数据库多使用B+树而不是B数：

1. 查询效率更好：B+树中，所有数据都是存在叶子结点的，并且叶子结点之间有一个链表连接，当需要查询某一范围内的数据时会更高效，因为一旦找到范围开始点，可以通过链表顺序访问所有相关的叶子结点。而在B数中，数据分布在整个树中，范围查询需要更多的磁盘访问。
2. 空间利用率：B+树的非叶子结点不存储数据，只存储键，因此可以存储更多的键，树的高度会更低。较低的树高度可以减少查找数据时需要读取的磁盘块的数据，提高了操作效率。
3. 维护成本：B+树在插入和删除操作时维护起来更加简单。由于所有数据都存储在叶子节点，并且非叶子结点仅存储键，所以对树的修改通常会引起更少的数据移动，维护成本更低。
4. 磁盘读写特性：数据库系统通常是基于磁盘的，而磁盘访问时间比内存访问时间要长得多。B+树更好地利用了磁盘块的特性，通过减少树的高度来减少磁盘读写次数。

<https://www.zhihu.com/question/635533248/answer/3393074991>

16. inner join, left join, right join, 和full join 之间的区别

INNER JOIN、LEFT JOIN、RIGHT JOIN 和 FULL JOIN 是 SQL 中用于结合两个或多个表的常用连接类型。它们之间的主要区别在于如何处理在连接条件中不匹配的记录。以下是这些连接类型的简要描述和区别：

INNER JOIN（内连接）

- 只返回两个表中满足连接条件的记录。
- 如果在其中一个表中没有与另一个表中的记录匹配的记录，那么这些记录不会出现在结果集中。
- 这是最常见的连接类型，因为它只返回两个表都有的数据。

LEFT JOIN（左连接）或 LEFT OUTER JOIN

- 返回左表中的所有记录，以及右表中与左表匹配的记录。
- 如果在右表中没有与左表中的记录匹配的记录，则结果集中对应的字段将为 NULL。
- 这意味着你可以确保从左表获取所有记录，并尝试获取与右表匹配的记录。

RIGHT JOIN（右连接）或 RIGHT OUTER JOIN

- 与 LEFT JOIN 相反，返回右表中的所有记录，以及左表中与右表匹配的记录。
- 如果在左表中没有与右表中的记录匹配的记录，则结果集中对应的字段将为 NULL。
- 需要注意的是，不是所有的数据库系统都支持 RIGHT JOIN。在不支持的系统上，你可以通过交换表的位置并使用 LEFT JOIN 来达到相同的效果。

FULL JOIN（全连接）或 FULL OUTER JOIN

- 返回左表和右表中的所有记录。
- 如果在其中一个表中没有与另一个表中的记录匹配的记录，则结果集中对应的字段将为 NULL。
- 这意味着你可以从两个表中获取所有记录，并尝试获取它们之间的匹配项。

多线程

1. 线程池

1.1. 线程池的作用

1. 降低资源消耗：通过池化技术重复利用已创建的线程，降低线程创建和销毁造成的损耗。
2. 提高响应速度：任务到达时，无需等待线程创建即可立即执行。
3. 提高线程的可管理性：线程是稀缺资源，如果无限制创建，不仅会消耗系统资源，还会因为线程的不合理分布导致资源调度失衡，降低系统的稳定性。使用线程池可以进行统一的分配、调优和监控。
4. 提供更多更强大的功能：线程池具备可拓展性，允许开发人员向其中增加更多的功能。比如延时定时线程池`ScheduledThreadPoolExecutor`，就允许任务延期执行或定期执行。

1.2. 线程池配置参数

建议使用`ThreadPoolExecutor`来创建线程池，同时尽量指定队列长度，避免取默认值，因任务太多导致内存溢出等问题。

不建议使用`Executors`来创建线程池，因为该构造方法没有可以设置队列长度的，可能发生内存溢出等问题。

对于CPU密集型的任务来说，核心线程数可以设置为机器核心数 + 1

对于IO密集型的任务来说，核心线程数可以设置为机器核心数 * 2 + 1

- `corePoolSize`: 核心线程数, 线程池在空闲时候会保活的线程数量
- `maximumPoolSize`: 线程池允许的最大线程数量
- `keepAliveTime, unit`: 线程保活的时间, 在当前线程数大于核心线程数的时候, 在定义的时间内, 超过数量的线程还是没任务执行的话, 会销毁
- `workQueue`: 线程池存放任务的队列
- `handler`: 在最大线程, 队列已经满了的时候, 再有任务进来的时候的处理策略。默认是直接抛弃

名称	描述
<code>ArrayBlockingQueue</code>	一个用数组实现的有界阻塞队列, 此队列按照先进先出(FIFO)的原则对元素进行排序。支持公平锁和非公平锁。
<code>LinkedBlockingQueue</code>	一个由链表结构组成的有界队列, 此队列按照先进先出(FIFO)的原则对元素进行排序。此队列的默认长度为 <code>Integer.MAX_VALUE</code> , 所以默认创建的该队列有容量危险。
<code>PriorityBlockingQueue</code>	一个支持线程优先级排序的无界队列, 默认自然序进行排序, 也可以自定义实现 <code>compareTo()</code> 方法来指定元素排序规则, 不能保证同优先级元素的顺序。
<code>DelayQueue</code>	一个实现 <code>PriorityBlockingQueue</code> 实现延迟获取的无界队列, 在创建元素时, 可以指定多久才能从队列中获取当前元素。只有延时期满后才能从队列中获取元素。
<code>SynchronousQueue</code>	一个不存储元素的阻塞队列, 每一个 <code>put</code> 操作必须等待 <code>take</code> 操作, 否则不能添加元素。支持公平锁和非公平锁。 <code>SynchronousQueue</code> 的一个使用场景是在线程池里。 <code>Executors.newCachedThreadPool()</code> 就使用了 <code>SynchronousQueue</code> , 这个线程池根据需要 (新任务到来时) 创建新的线程, 如果有空闲线程则会重复使用, 线程空闲了60秒后会被回收。
<code>LinkedTransferQueue</code>	一个由链表结构组成的无界阻塞队列, 相当于其它队列, <code>LinkedTransferQueue</code> 队列多了 <code>transfer</code> 和 <code>tryTransfer</code> 方法。
<code>LinkedBlockingDeque</code>	一个由链表结构组成的双向阻塞队列。队列头部和尾部都可以添加和移除元素, 多线程并发时, 可以将锁的竞争最多降到一半。

	名称	描述
1	<code>ThreadPoolExecutor.AbortPolicy</code>	丢弃任务并抛出 <code>RejectedExecutionException</code> 异常。这是线程池默认的拒绝策略, 在任务不能再提交的时候, 抛出异常, 及时反馈程序运行状态。如果是比较关键的业务, 推荐使用此拒绝策略, 这样子在系统不能承载更大的并发量的时候, 能够及时的通过异常发现。
2	<code>ThreadPoolExecutor.DiscardPolicy</code>	丢弃任务, 但是不抛出异常。使用此策略, 可能会使我们无法发现系统的异常状态。建议是一些无关紧要的业务采用此策略。
3	<code>ThreadPoolExecutor.DiscardOldestPolicy</code>	丢弃队列最前面的任务, 然后重新提交被拒绝的任务。是否要采用此种拒绝策略, 还得根据实际业务是否允许丢弃老任务来认真衡量。
4	<code>ThreadPoolExecutor.CallerRunsPolicy</code>	由调用线程 (提交任务的线程) 处理该任务。这种情况是需要让所有任务都执行完毕, 那么就适合大量计算的任务类型去执行, 多线程仅仅是增大吞吐量的手段, 最终必须要让每个任务都执行完毕。

1.3. 线程池的执行流程

workerNum: 当前正在工作的线程

1. 任务一个个接着过来, 创建线程直到达到核心线程数。
2. 达到workerNum == corePoolNum后, 如果workQueue还没满的话, 把任务加入到workQueue中
3. workQueue满了的话, 创建非核心线程, 随着任务增加, 直到workerNum == maximumPoolSize
4. 这个时候达到了最大线程数, 如果还有任务过来的话, 则根据配置的拒绝策略进行处理, 默认是直接抛弃并且报错 RejectedExecutionException

1.4. 一些线程池

1.4.1. SingleThreadPool

单个线程的线程池, corePoolSize和maximumPoolSize都设置为1, 使用无界队列 LinkedBlockingQueue, 可以保证顺序执行, 适用于串行执行任务的场景。

1.4.2. FixedThreadPool

corePoolSize和maximumPoolSize相等, 使用无界队列 LinkedBlockingQueue。

线程数量固定的线程池, 当线程处于空闲状态时, 不会被回收, 除非线程池关闭;

当所有的线程都处于活动状态时, 新的任务都会处于等待状态, 直到线程池空闲;

使用于CPU密集型的任务, 确保CPU在长期被工作线程使用的情况下, 尽可能的少的分配线程, 即适用执行长期的任务。

1.4.3. ScheduledThreadPool

```
public static ScheduledExecutorService newScheduledThreadPool(int corePoolSize) {  
    return new ScheduledThreadPoolExecutor(corePoolSize);  
}  
  
public ScheduledThreadPoolExecutor(int corePoolSize) {  
    super(corePoolSize, Integer.MAX_VALUE, 0, NANOSECONDS,  
        new DelayedWorkQueue());  
}
```

线程总数阈值为Integer.MAX_VALUE,工作队列使用DelayedWorkQueue，非核心线程存活时间为0，所以线程池仅仅包含固定数目的核心线程。

适用于周期性执行任务，并且需要限制线程数量的场景。

<https://blog.csdn.net/liuchangjie0112/article/details/90698401>

1.4.4. CachedThreadPool

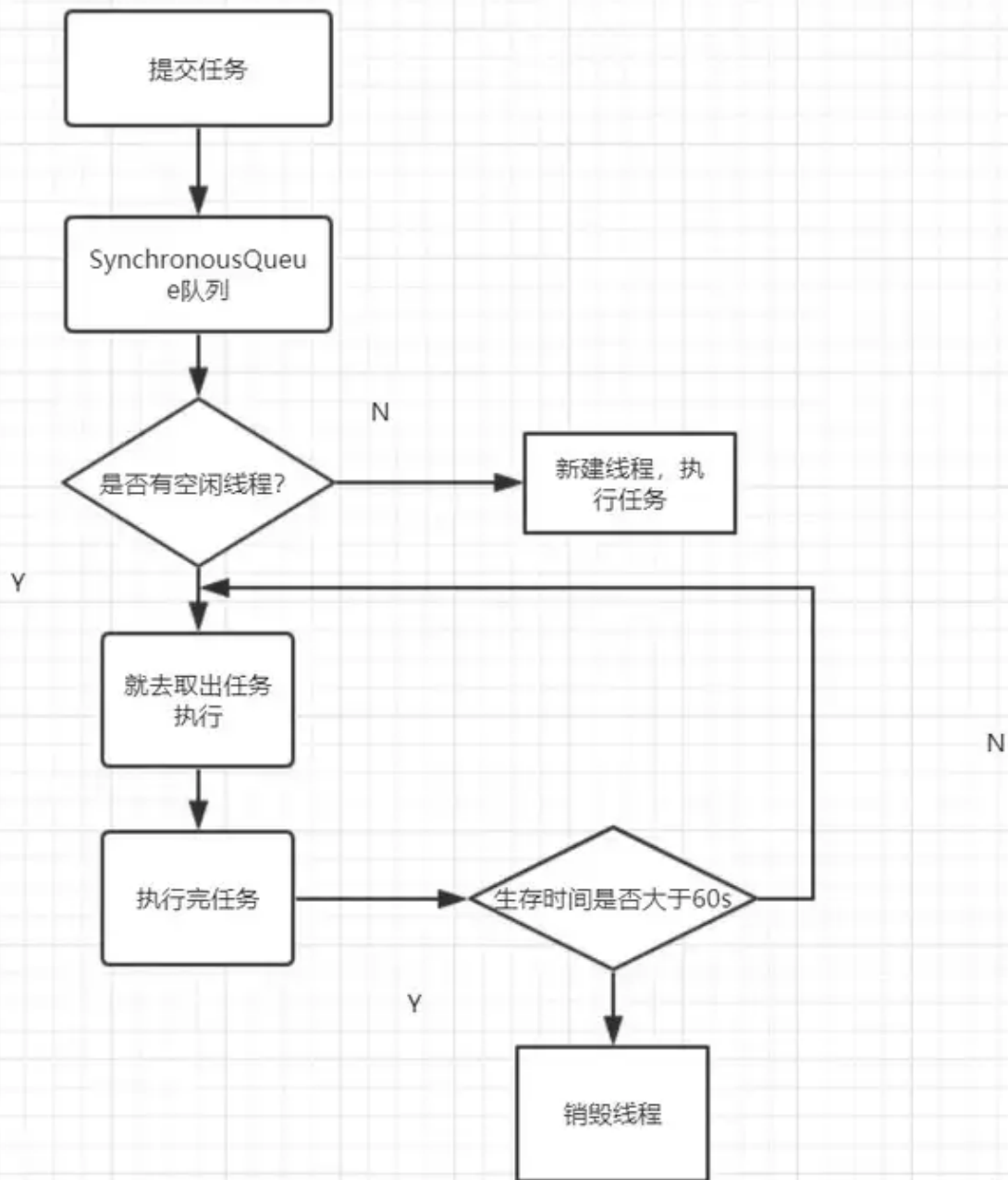

```
public static ExecutorService newCachedThreadPool() {  
    return new ThreadPoolExecutor(0, Integer.MAX_VALUE,  
        60L, TimeUnit.SECONDS,  
        new SynchronousQueue<Runnable>());  
}
```

核心线程数为0，总线程数量阈值为Integer.MAX_VALUE, SynchronousQueue做为队列，即可以创建无限的非核心线程

执行流程

1. 先执行SynchronousQueue的offer方法提交任务，并查询线程池中是否有空闲线程来执行SynchronousQueue的poll方法来移除任务。如果有，则配对成功，将任务交给这个空闲线程
2. 否则，配对失败，创建新的线程去处理任务
3. 当线程池中的线程空闲时，会执行SynchronousQueue的poll方法等待执行SynchronousQueue中新提交的任务。若等待超过60s,空闲线程就会终止

适用于执行大量短生命周期任务，且这些任务的执行时间远小于线程创建和销毁的时间开销。因为maximumPoolSize是无界的，所以提交任务的速度 > 线程池中线程处理任务的速度就要不断创建新线程；每次提交任务，都会立即有线程去处理，因此CachedThreadPool适用于处理大量、耗时少的任务。



知乎 @老刘

<https://zhuanlan.zhihu.com/p/132748927>

1.5. 扩展

线程池的最大线程数设置太小，容易造成很多拒绝任务，服务降级；

队列设置太长，请求大量增加的时候，也可能造成任务堆积，导致下游服务超时等；

很难基于业务判断，可以考虑使用动态线程池，通过把线程池的配置参数放到分布式配置中心去，同时加上监控告警等。

<https://tech.meituan.com/2020/04/02/java-pooling-practice-in-meituan.html>

2. ThreadLocal

2.1. ThreadLocal 与 Synchronized

1. 都是来解决多线程下的并发访问
2. Synchronized 用于线程间的数据共享，所有线程都能访问到共享变量；
3. ThreadLocal 用于线程间的数据隔离，每个线程访问的是变量的一个副本，不同的对象；

2.2. ThreadLocal && InheritableThreadLocal

Thread类中成员变量，ThreadLocalMap

ThreadLocalMap中的是Entry数组的，table

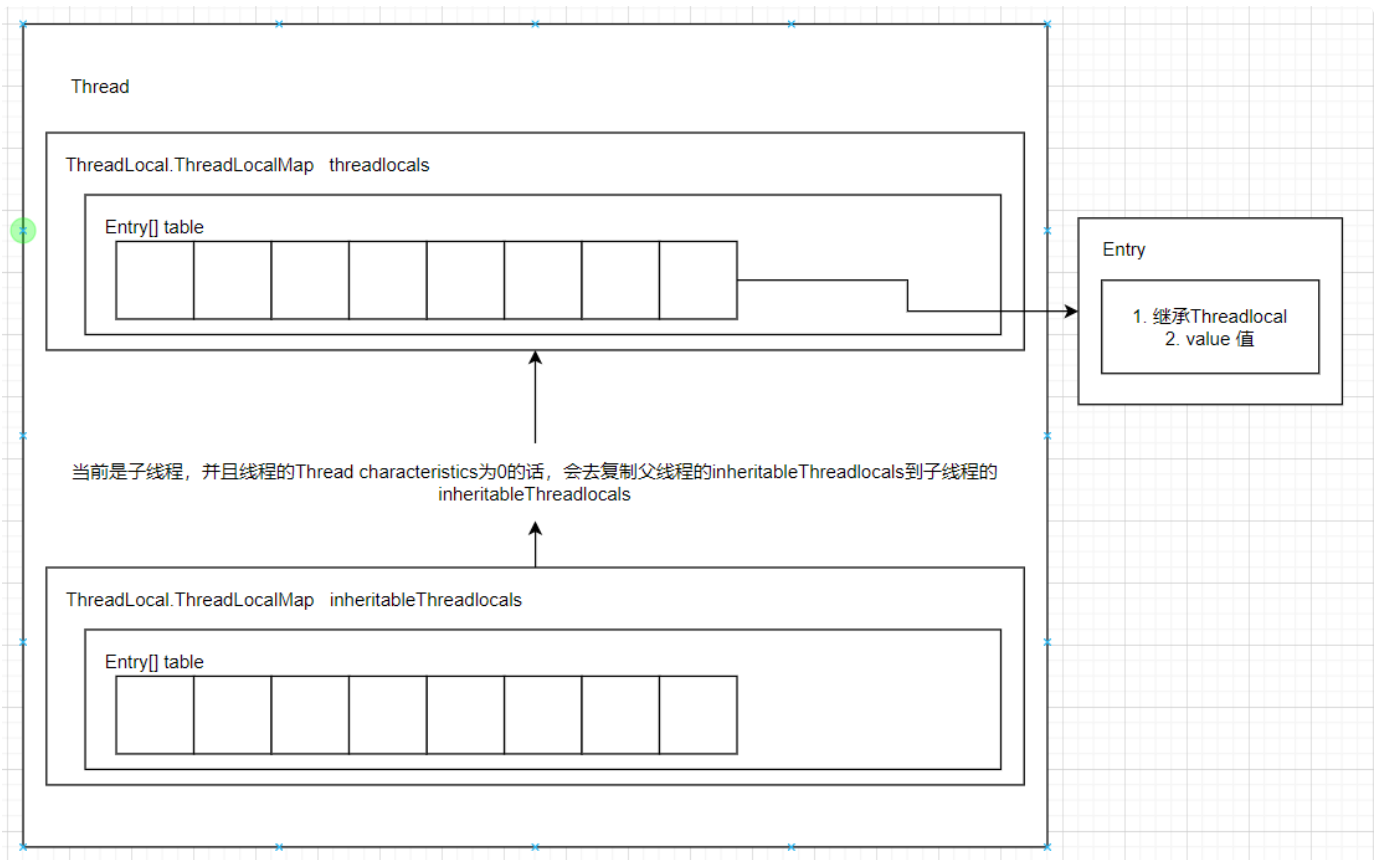
Entry继承了ThreadLocal，并且添加了成员变量value，用来存变量在每个线程中副本的值

通过threadlocal变量的threadLocalHashCode，与table的容量（默认是16）进行与运算，得到下标索引，把值存入对应索引的位置。

threadLocalHashCode是原子整型，线程安全，大概是每次加上0x61c88647。

线程在创建的时候，会去判断当前是子线程，并且线程的Thread characteristics为0的话，会去复制父线程的inheritableThreadlocals 到子线程的 inheritableThreadlocals

threadlocalmap初始容量为16，在达到阈值2/3后，会自动扩容。这个时候先申请原来两倍长度的数组，然后再把原来的entry重新取与运算之后放入table中，这个过程中，会把Key为null的对应entry的值也设置为null，尝试进行垃圾回收。



2.3. ThreadLocal 的使用场景

使用完，要手动调用remove()。

使用于如下两种场景

1. 每个线程需要有自己单独的实例
2. 实例需要在多个方法中共享，但不希望被多线程共享

详细场景如：

1. 使用日期工具类，当用到SimpleDateFormat，可以保证线程安全
2. 全局存储信息，如存入用户信息，那么当前线程在任何需要的地方都可以使用
3. 保证同一个线程，获取的数据库连接Connection是同一个，使用ThreadLocal来解决线程安全问题
4. 使用MDC保存日志信息

https://blog.csdn.net/qq_43842093/article/details/126715922

https://blog.csdn.net/gongzi_9/article/details/126754648

2.4. ThreadLocal 存在的问题

内存泄漏：

ThreadLocalMap使用ThreadLocal的弱引用作为key，当ThreadLocal变量被手动设置为null，即一个ThreadLocal没有外部强引用来引用它，当系统GC时，ThreadLocal一定会被回收。这样的话，ThreadLocalMap中就会出现key为null的Entry，就没有办法访问这些key为null的Entry的value，如果当前线程再迟迟不结束的话(比如线程池的核心线程)，这些key为null的Entry的value就会一直存在一条强引用链：Thread变量 -> Thread对象 -> ThreadLocalMap -> Entry -> value -> Object 永远无法回收，造成内存泄漏。

缓存

<https://mp.weixin.qq.com/s/Vu7Fgy-P1XozFIFAURByMQ>

1. 缓存穿透

请求的数据既不在缓存中，也不在数据库中，每一次请求过来都需要查询一次数据库，可能会直接打挂掉数据库

解决方法：不要让每一次请求都直接去请求数据库，过滤掉一些输入。

具体方法：

- 1) 校验参数。校验掉恶意伪造的请求数据
- 2) 使用布隆过滤器。把数据库的数据通过多次hash映射到布隆过滤器的bit上，在布隆过滤器上的才进行后续查询操作，不在的就直接返回
- 3) 缓存空值。对于请求过来的，缓存和数据库都没有的数据也缓存起来，值为空。

2. 缓存击穿

大量用户同时请求一个数据，但是这个数据的缓存正好失效了，这就会造成很多请求都怼到了数据库，造成瞬间数据库压力过大，挂掉。

解决方法: 避免在一个时间段内，很多请求同时请求某个数据。

具体方法:

- 1) 加锁。
- 2) 用定时任务为缓存的过期时间延长
- 3) 把热点缓存数据设置为永不过期

3. 缓存雪崩

有一段时间，大量用户请求多个数据，但是这些数据的缓存正好在这一时间段内都过期了，也会造成很多数据直接请求数据库，导致数据库挂掉。

解决办法: 避免很多缓存数据在同一时刻过期

具体方法:

- 1) 过期时间添加随机数
- 2) 保证高可用。用redis集群或者是哨兵模式等。
- 3) 服务降级。在高可用的情况下，还是会挂掉的话，考虑用服务降级，有兜底的方法和数据。

比如: 程序中有个全局开关，比如有10个请求在最近一分钟内，从redis中获取数据失败，则全局开关打开。后面的新请求，就直接从配置中心中获取默认的数据。

当然，还需要有个job，每隔一定时间去从redis中获取数据，如果在最近一分钟内可以获取到两次数据（这个参数可以自己定），则把全局开关关闭。后面来的请求，又可以正常从redis中获取数据了。

4. 数据库与缓存双写一致性

https://mp.weixin.qq.com/s?__biz=MzkwNjMwMTgzMQ==&mid=2247493521&idx=1&sn=bff84e7a819d79e4b8eb7e722e96ddfc&chksm=c0e83f79f79fb66f2e7bf03a104580b404ea0a3c977846e428f13c1f12fbad46d4d778b2da14&token=115145471&lang=zh_CN&scene=21#wechat_redirect

总结:

低并发下，且写数据库和缓存在一个事务内，可以用先写数据库再写缓存

高并发下，可以先写数据库，再删缓存。

1) 先写缓存再写数据库:

当写完缓存后, 因网络等原因, 写数据库失败, 缓存不一致。最严重。

2) 先写数据库再写缓存:

当写完数据库之后, 当写缓存失败,

a. 假如两个操作都在同一个事务时, 可以回滚, 适用于低并发的场景。

b. 但业务需要, 或者需要避免大事务等原因, 不在一个事务时, 不能够回滚, 出现数据库和缓存不一致问题。(单个请求)

c. 高并发场景中, 如果多个线程同时执行先写数据库, 再写缓存的操作, 可能会出现数据库是新值, 而缓存中的是旧值。(如写线程a, 更新数据库之后, 网络卡顿, 还没来得及更新缓存。这个时候写线程b, 更新数据库, 写入缓存成功。但是线程a网络恢复, 写缓存, 会把线程b的缓存覆盖掉, 会导致线程b的缓存和数据库两边不一致)

3) 先删缓存再写数据库:

a. 高并发场景下, 也会出现不一致的问题。(如写线程c, 先删除缓存, 网络卡顿, 还没来得及写入数据库。这时候, 读线程d, 发现缓存没数据, 去查数据库有数据, 旧值, 写入到缓存, 返回。这时候写线程c结束卡顿, 开始写入数据库, 就会造成两边数据不一致了。)

b. 上面的这个问题, 可以在写线程c写入数据库之后, 过一段时间再删除缓存, 即是缓存双删。注意得隔一段时间, 因为需要确保读线程d把数据库旧值写入到缓存之后再删, 确保删除旧值。

c. 如果第二次删除失败的话, 可以用重试机制。

同步重试

异步重试: 1. mq 2. 分布式定时任务elastic-job 3. 订阅mysql的binlog, 在订阅者中, 如果发现了更新数据请求, 则删除相应的缓存

4) 先写数据库再删缓存:

a. 高并发场景中，读请求和写请求那个先过来都行。

场景1. 读请求先过来。

读线程e过来，发现缓存有数据，直接返回。

写线程f过来，写数据库，删除缓存。

场景2. 写请求先过来

写请求f过来，写数据库，网络卡顿，没来得及删除缓存

读请求e过来，发现有缓存，获取后直接返回

写请求恢复，删除缓存，及时删除了缓存

b. 当在一种情况下还是会出现不一致的情况

缓存过期时间到了，自动失效

请求f查询缓存，没数据，查询数据库有数据，是旧值，但网络卡顿，没来得及更新缓存

请求e先写数据库，接着删除缓存

请求f更新旧值到缓存中，出现了两边不一致的问题

需要满足条件，缓存自动失效，请求f从数据库查出旧值，更新缓存的耗时比请求e写数据库，删除缓存的时间还长。条件成立的概率比较小。

5. 大key问题

key存储的数据随着时间业务演变，变得很大

解决方法:

a. 缩减字段名称。存的是json格式的值，可以缩减大量出现的重复字段名称。

b. 压缩。把存储的数据压缩成byte数组。

6. 热key问题

二八原理：80%的用户经常访问20%的热点数据。

大量的用户请求集中访问同一天Redis服务器节点，该节点很有可能会因为扛不住这么大的压力，而直接down机。

解决方法：

a. 拆分key

评估后，将热点key分开保存，不要集中保存到同一台Redis服务器节点。

b. 增加本地缓存

对应热key，可以增加一层本地缓存，能够提升性能的同时也能避免Redis访问量过大的问题。

7. 集群下，本地缓存一致性的解决方案：

一致性指的是在同时使用缓存和数据库的场景下，要确保数据在缓存与数据库中的更新操作保持同步，当对数据进行更改时，无论是先修改缓存还是先修改数据库，最终都要保持两者的数据是一致的，不会出现数据不一致的问题。

在分布式系统中，每台机器都有自己本地的缓存，要保证一致性是比较难的。

解决方案如下：

1. 设置本地缓存短时间内失效。

- 短的存活周期，保证了数据的时效性比较高，当数据失效之后，再次访问数据就会拉取最新数据，能尽可能的保持数据一致性。
- 代码实现简单，不需要写多余代码；缺点是效果不明显，不适合高并发系统。

2. 通过配置中心来协调和同步。

- 通过微服务中的配置中心（如nacos）来协调，因为所有服务器都会连接到配置中心。
- 所以当数据修改之后，可以修改配置中心的配置，然后配置中心再把配置变更的事件推送给各个服务，各个服务感知到配置中心的配置发生变更之后，再更新自己的本地缓存，这样就实现了本地缓存的数据一致性。

3. 本地缓存自带更新功能。

- 使用本地缓存框架的自动更新功能，例如Caffeine的refresh的功能来自动刷新缓存，这样就可以设置很短时间来更新最新的数据，从而也能尽可能保证数据的一致性。

```
1 // 创建 Caffeine 缓存实例
2 Cache<String, String> caffeineCache = Caffeine.newBuilder()
3 // 设置缓存项在 5s 后开始自动更新
4 .refreshAfterWrite(5, TimeUnit.SECONDS)
5 // 自定义缓存更新逻辑（即获取新值逻辑）
6 .build(new CacheLoader<String, String>() {
7     @Override
8     public void reload(String key, String oldValue) throws Exception {
9         // 模拟更新缓存的操作
10        updateCache(key, oldValue);
11    }
12 });
```

结论：

- 如果对数据一致性要求不高，并且并发压力不大的情况下，可以使用设置本地缓存短时间内失效的解决方案。
- 如果对数据的一致性要求极高，且有配置中心的情况下，可使用配置中心协调和同步本地缓存。
- 如果对一致性要求不高，且为高并发系统，可以采用本地缓存的自动更新功能来实现。

分布式相关

1. 分布式锁

<https://cloud.tencent.com/developer/article/1197181?areald=106001>

分布式锁：

控制分布式系统或不同系统之间共同访问共享资源的一种锁实现。如果不同的系统或同一个系统的不同主机之前共享了某个资源时候，往往需要互斥来防止彼此干扰来保证一致性。

分布式锁需要具备的条件：

互斥性：在任意一个时刻，只有一个客户端持有锁

无死锁：即便持有锁的客户端崩溃或者其他意外事件，锁然后可以被获取

容错：只要大部分Redis节点都活着，客户端就可以获取和释放锁

1.1. Redis分布式锁

单机Redis实现分布式锁：

1. Redis指令：SET key value NX EX 10

- NX ---- 不存在键则添加键 EX ---- 当且仅当存在键值时，才设置键；
- EX ---- 过期时间，秒 PX ---- 过期时间，毫秒；
- 在设置键的同时，也设置过期时间，避免客户端挂了之后，锁永远不释放；
- 键的值value，需要为唯一值，避免因网络等原因，锁过期，完成业务操作的线程误释放其他线程的锁；
- 可以自动延长锁过期时间，来避免业务操作时间长于锁过期时间的场景；

2. 删除锁的操作需要原子性，并且不能误删其他线程的锁

- 脚本: `if redis.call('get', KEYS[1]) == ARGV[1] then return redis.call('del', KEYS[1]) else return 0 end;`

集群Redis实现分布式锁： RedLock

加锁步骤：

1. 获取当前Unix时间，以毫秒为单位。
2. 依次尝试从N个实例，使用相同的key和随机值获取锁。在步骤2，当向Redis设置锁时，客户端应该设置一个网络连接和响应超时时间，这个超时时间应该小于锁的失效时间。例如你的锁自动失效时间为10秒，则超时时间应该在5-50毫秒之间。这样可以避免服务器端Redis已经挂掉的情况下，客户端还在死死地等待响应结果。如果服务器端没有在规定时间内响应，客户端应该尽快尝试另外一个Redis实例。
3. 客户端使用当前时间减去开始获取锁时间（步骤1记录的时间）就得到获取锁使用的时间。当且仅当从大多数（这里是3个节点）的Redis节点都取到锁，并且使用的时间小于锁失效时间时，锁才算获取成功。
4. 如果取到了锁，key的真正有效时间等于有效时间减去获取锁所使用的时间（步骤3计算的结果）。
5. 如果因为某些原因，获取锁失败（没有在至少 $N/2+1$ 个Redis实例取到锁或者取锁时间已经超过了有效时间），客户端应该在所有的Redis实例上进行解锁（即便某些Redis实例根本就没有加锁成功）。

解锁步骤：

向所有的Redis实例发送释放锁命令即可，不用关心之前有没有从Redis实例成功获取到锁。

Redis工具

- Jedis是Redis官方推出的用于通过Java连接Redis客户端的一个工具包，提供了Redis的各种命令支持
- Lettuce是一种可扩展的线程安全的 Redis 客户端，通讯框架基于Netty，支持高级的 Redis 特性，比如哨兵，集群，管道，自动重新连接和Redis数据模型。 Spring Boot 2.x 开始 Lettuce 已取代 Jedis 成为首选 Redis 的客户端。
- Redisson是架设在Redis基础上，通讯基于Netty的综合的、新型的中间件，企业级开发中使用Redis的最佳范本

建议使用Redisson

1.2. zookeeper分布式锁

<https://zhuanlan.zhihu.com/p/639756647>

1. 在zookeeper上创建一个父节点，临时顺序节点。
2. 第一个线程进来之后，在父节点下创建子节点，为XXXX0001号节点。接着拿父节点下所有节点，发现本身是最小的，视为加锁成功。
3. 在第一个线程还没释放锁的时候，第二个线程进来了，也在父节点下创建子节点，为XXXX0002号节点。遍历所有节点，发现XXXX0001才是最小的，则在XXXX0001节点上加监听事件，继续等待。
4. 第一个线程执行完成之后，删除XXXX0001节点，释放锁。这时候，zk会触发监听事件，告诉XXXX0002它前面的节点释放锁了或者是断开连接了等。
5. 第二个线程再一次遍历所有节点，发现本身是最小的了，加锁成功。

临时节点可以防止死锁问题（锁不释放），因为当失去连接后，zk会自动删除临时节点。

节点排序监听可以提供阻塞功能，监听上一个节点可以解决羊群效应（因为假如不是监听上个节点，而是每个节点都去监听拿到锁的节点的话，当锁释放的时候，所有的节点都会去抢夺锁，随时可能干废机器。）

1.3. 数据库分布式锁

悲观锁：额外维护一个表，多线程情况下，只有插入成功的表，才视为加锁成功

乐观锁：在表上再加一个版本号的字段，先查出当前操作的记录的版本号，然后在要进行操作的时候，比较下版本号和数据库记录的版本号是否一样，一样才进行操作。

2. 分布式事务

2.1. 场景

- 跨JVM进程产生分布式事务；如订单服务下单，操作订单数据库，并且下单的服务中调库存服务，操作库存数据库，减少库存。
- 跨数据库实例产生分布式事务；如订单服务下单，操作订单数据库，插入订单数据，同时操作客户数据库，新增一个用户。
- 多个服务同时访问一个数据库实例；比如订单服务和库存服务同时访问同一个数据库，原因是跨JVM进程，两个微服务有了不同的数据库连接进行数据库操作，此时产生了分布式事务。

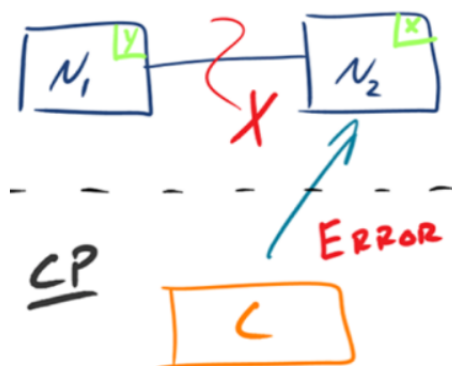
多个微服务操作至少一个数据库，不加分布式事务的话，可能存在订单插入，库存没减少等的数据库不一致问题。

2.2. CAP理论

1. CAP 定理 (CAP theorem) 又被称作布鲁尔定理 (Brewer's theorem)，是加州大学伯克利分校的计算机科学家埃里克·布鲁尔 (Eric Brewer) 在 2000 年的 ACM PODC 上提出的一个猜想。2002 年，麻省理工学院的赛斯·吉尔伯特 (Seth Gilbert) 和南希·林奇 (Nancy Lynch) 发表了布鲁尔猜想的证明，使之成为分布式计算领域公认的一个定理。对于设计分布式系统的架构师来说，CAP 是必须掌握的理论。
2. 简单来说：在一个分布式系统（指互相连接并共享数据的节点的集合）中，当设计读写操作时，只能够保证一致性、可用性、分区容错性三者中的两个，牺牲另外一个。
 - 一致性 (Consistency) 对某个指定的客户端来说，读操作保证能够返回最新的写操作结果。
 - 可用性 (Availability) 非故障的节点在合理的时间内返回合理的响应，不是错误和超时的响应。
 - 分区容忍性 (Partition Tolerance) 当出现网络分区（即部分节点之间的通信出现故障或延迟）后，系统能够继续运行并保持数据一致性的能力。
3. 虽然CAP理论是三个要素只能取两个，但是放到实际分布式环境下来思路，会发现P要素必须选择，因为网络本身无法做到100%可靠，有可能出现故障，所有分区时一个必然的现象。假如我们选择了CA而放弃了P，那么当发生分区现象时，为了保证C，系统需要禁止写入，当有写入请求时，系统返回error（当前系统不允许写入等），这又和A要素冲突，因为A要求返回 no error 和 no timeout。因此，分布式系统理论上不可能选择CA架构，只能选择CP或AP架构。

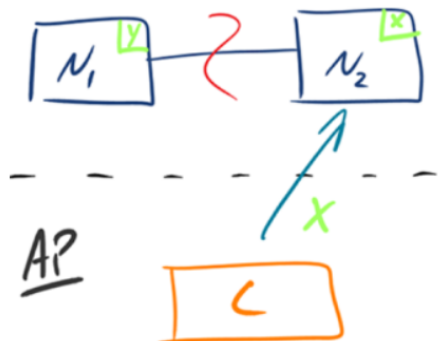
CP - Consistency/Partition Tolerance

如下图所示，为了保证一致性，当发生分区现象后，N1 节点上的数据已经更新到 y，但由于 N1 和 N2 之间的复制通道中断，数据 y 无法同步到 N2，N2 节点上的数据还是 x。这时客户端 C 访问 N2 时，N2 需要返回 Error，提示客户端 C“系统现在发生了错误”，这种处理方式违背了可用性 (Availability) 的要求，因此 CAP 三者只能满足 CP。



AP - Availability/Partition Tolerance

如下图所示，为了保证可用性，当发生分区现象后，N1 节点上的数据已经更新到 y，但由于 N1 和 N2 之间的复制通道中断，数据 y 无法同步到 N2，N2 节点上的数据还是 x。这时客户端 C 访问 N2 时，N2 将当前自己拥有的数据 x 返回给客户端 C 了，而实际上当前最新的数据已经是 y 了，这就不满足一致性（Consistency）的要求了，因此 CAP 三者只能满足 AP。注意：这里 N2 节点返回 x，虽然不是一个“正确”的结果，但是一个“合理”的结果，因为 x 是旧的数据，并不是一个错乱的值，只是不是最新的数据而已。



综合上面，一般对于数据实时性不是很严格的话，都会采用AP模式，以BASE理论中的最终一致性来代替CAP中的强一致性A。

2.3. BASE理论

1. BASE指基本可用（Basically Available）、软状态（Soft State）、最终一致性（Eventual Consistency），核心思想是即使无法做到强一致性，CAP中的C，但应用可以采用合适的方式达到最终一致性。满足BASE理论的事务，也称为"柔性事务"。

- 基本可用性：在分布式系统出现故障时，允许损失部分可用功能，保证核心功能可用。如电商网站交易付款出现问题，商品依然可以正常浏览。
- 软状态：由于不要求强一致性，所有BASE允许系统重存在中间状态，软状态，这个状态不影响系统可用性，如订单的支付中，数据同步中等状态，待数据最终一致后状态改为成功。
- 最终一致性：指经过一段时间后，所有节点数据都将会达到一致。如订单的支付中状态，最终会变为支付成功或者支付失败，使订单状态和实际交易结果达成一致，但需要一定时间的延迟，等待。

2. 理解强一致性和最终一致性

CAP理论告诉我们一个分布式系统最多只能同时满足一致性（Consistency）、可用性（Availability）和分区容忍性（Partition tolerance）这三项中的两项，其中AP在实际应用中较多，AP即舍弃一致性，保证可用性和分区容忍性，但是在实际生产中很多场景都要实现一致性，比如前边我们举的例子主数据库向从数据库同步数据，即使不要一致性，但是最终也要将数据同步成功来保证数据一致，这种一致性和CAP中的一致性不同，CAP中的一致性要求在任何时间查询每个结点数据都必须一致，它强调的是强一致性，但是最终一致性是允许可以在一段时间内每个结点的数据不一致，但是经过一段时间每个结点的数据必须一致，它强调的是最终数据的一致性。

2.4. 分布式事务解决方案

2.4.1. 2PC，二阶段提交

2.4.1.1. 2PC

2PC, Two-phase commit protocol, 二阶段提交。是一种强一致性设计。

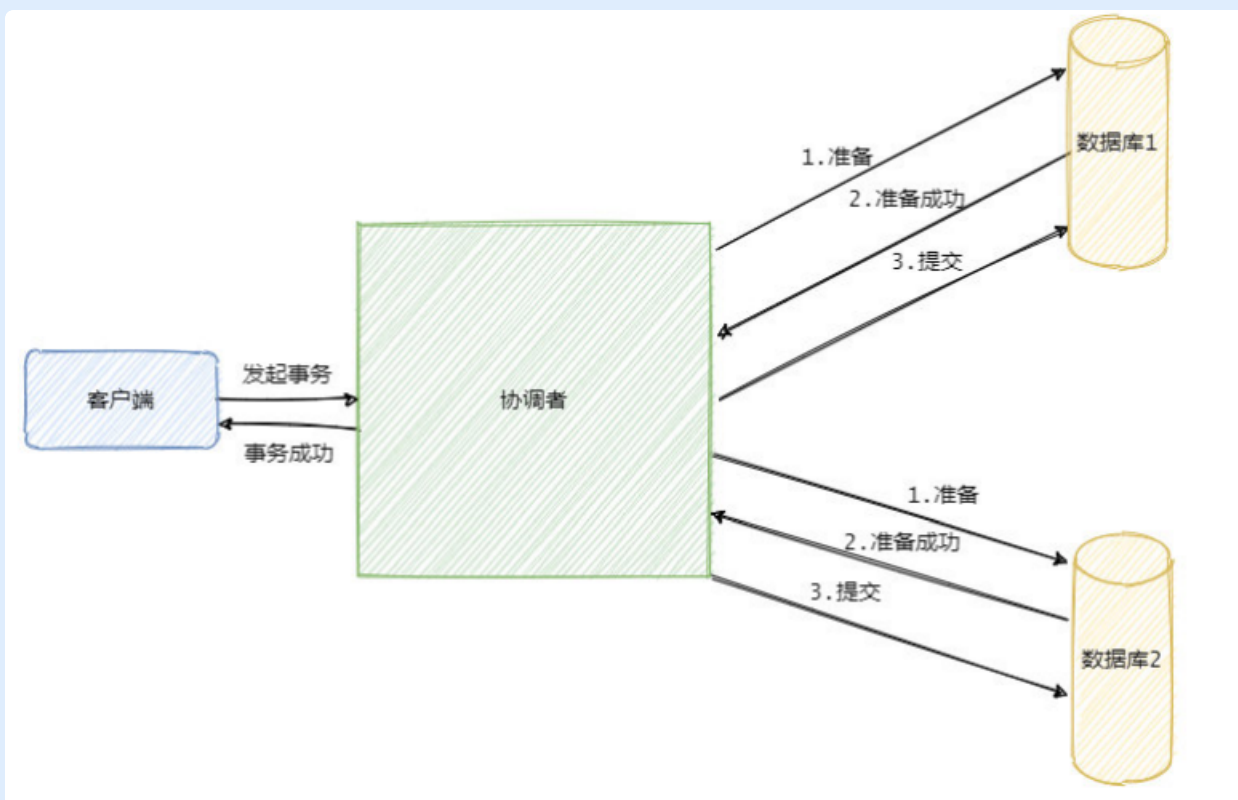
引入了一个事务协调者的角色来协同管理各参与者（本地资源）的提交和回滚，二阶段指的是准备（投票）和提交两个阶段。

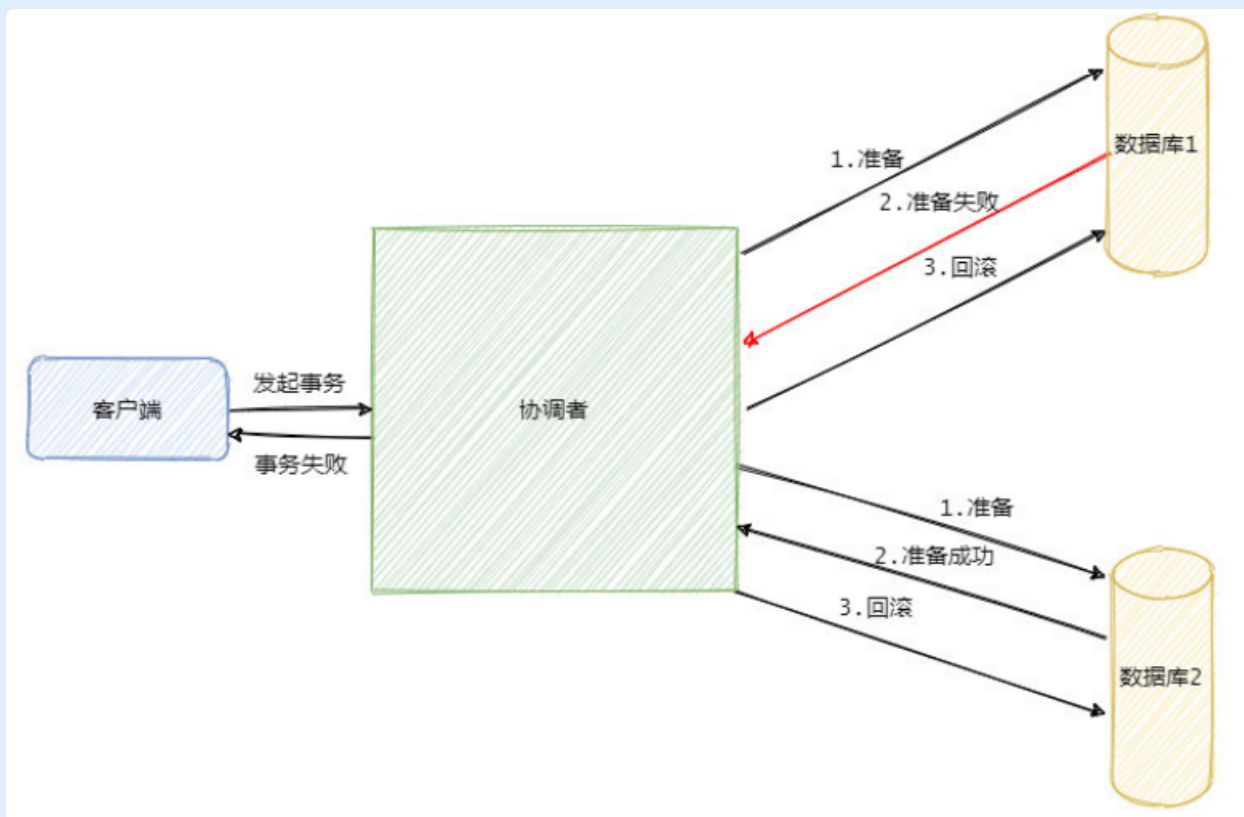
1. 准备阶段：事务管理器给每个参与者发送Prepare消息，每个数据库参与者在本地执行，并写本地的Undo/Redo 日志，这是还没提交事务。（Undo日志是记录修改前的数据，用于数据库回滚，Redo日志是记录修改后的数据，用于提交事务后写入数据文件）
2. 提交事务：如果事务管理器收到了参与者的执行失败或超时消息时，直接给每个参与者发送回滚消息；否则发送提交消息；参与者根据事务管理器的指令执行提交或回滚操作，并释放事务处理过程中使用的锁资源。

2PC是同步阻塞协议，需要等第一阶段的所有参与者都响应之后，才能进行下一步操作。

同步阻塞会导致长时间的资源锁定问题，效率低，并且存在单点故障问题，在极端条件下存在数据不一致问题。

2PC适用于数据库层面的分布式事务场景。





第二阶段执行提交或者回滚事务失败了的话，只能够不断重试，到最后实在不行则只能人工介入。

协调者是单节点的，存在单点故障问题：

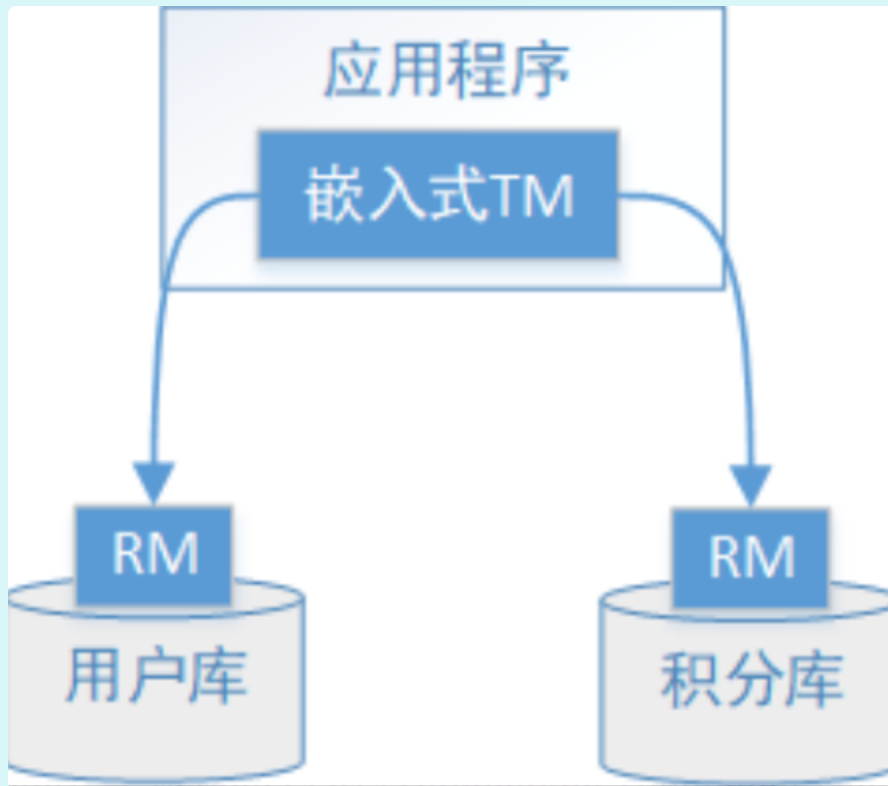
- 假如协调者在发送准备之前挂了，相当于事务还没开始。
- 假如协调者在发送准备之后挂了，在准备阶段参与者执行了，事务资源锁定了。但是进入不了提交阶段，事务执行不下去，可能会因为锁定了一些公共资源而阻塞系统其他操作。
- 假如协调者在发送回滚事务命令之前挂了，事务也是执行不下去，且第一阶段那些准备成功的参与者都阻塞着。
- 假如协调者在发送回滚事务命令之后挂了，至少命令发出去了，很大概率会回滚成功，资源释放。但是如果因为出现网络分区问题，某些参与者因收不到指令而阻塞着。
- 假如协调者在发送提交事务之前挂了，所有资源都阻塞了。
- 假如协调者在发送提交事务之后挂了，至少命令发出去了，很大概率会提交成功，资源释放。但是如果因为出现网络分区问题，某些参与者因收不到指令而阻塞着。

协调者故障，可以通过选举得到新协调者：

1. 如果这个时候处于准备阶段，新协调者发出回滚事务命令则行。
2. 如果处于提交阶段，
 - a. 假设参与者都没挂，此时新协调者可以向所有参与者确认下他们自身情况下推断下一步操作。
 - b. 假如有个别参与者挂了，这时候就有点僵硬。
 - b1) 比如协调者发送回滚指令，此时第一个参与者收到并且执行，然后协调者和第一个参与者就都挂了的话，其他参与者都还没收到请求。然后新协调者来了，它确认其他参与者都可以提交，但它不知道挂了的那个参与者到底可不可以提交，所有僵持住了，懵了。
 - c1) 这种情况下，就算协调者知道自己该发起提交请求，那么也没用，因为不知道挂掉的那个参与者提交了事务没。如果参与者在挂之前提交事务成功，新协调者确认还在的参与者没问题，那发起提交事务指令就可以保证数据一致了。但参与者在挂之前事务还没提交成功，参与者在恢复之后数据是回滚的，那新协调者这时只能发起回滚事务才能保持事务一致性。所以说极端情况下还是无法避免数据不一致问题。

2.4.1.2. XA协议

2PC的传统方案是在数据库层面实现的，如Oracle、MySQL都支持2PC协议，为了统一标准减少行业内不必要的对接成本，需要制定标准化的处理模型及接口标准，国际开放标准组织Open Group定义了分布式事务处理模型DTP（Distributed Transaction Processing Reference Model）。



和上述2PC的流程基本一致，补充了概念：AP -> 应用程序；事务管理者 -> TM；参与者 -> RM；

- AP(Application Program)：即应用程序，可以理解为使用DTP分布式事务的程序。
- RM(Resource Manager)：即资源管理器，可以理解为事务的参与者，一般情况下是指一个数据库实例，通过资源管理器对该数据库进行控制，资源管理器控制着分支事务。
- TM(Transaction Manager)：事务管理器，负责协调和管理事务，事务管理器控制着全局事务，管理事务生命周期，并协调各个RM。全局事务是指分布式事务处理环境中，需要操作多个数据库共同完成一个工作，这个工作即是一个全局事务。
- DTP模型定义TM和RM之间通讯的接口规范叫XA，简单理解为数据库提供的2PC接口协议，基于数据库的XA协议来实现2PC又称为XA方案。

以上三个角色之间的交互方式如下：

1) TM向AP提供 应用程序编程接口，AP通过TM提交及回滚事务。

2) TM交易中间件通过XA接口来通知RM数据库事务的开始、结束以及提交、回滚等。

2.4.1.3. Seata分布式事务框架

Seata 是阿里中间件团队发起的开源项目 Fescar，后更名为 Seata，它是一个开源的分布式事务框架。

解决了传统2PC的问题，通过对本地关系数据的分支事务的协调来驱动完成全局事务，是工作在应用层的中间件。主要优点是性能较好，且长时间不占用连接资源，以高效并且对业务0侵入的方式解决微服务场景下面临的分布式事务问题，目前提供AT模式（即2PC）以TCC 模式的分布式事务解决方案。

一个ID+三组件模型（1+3）：

一个全局唯一的事务ID。

TC-事务协调者：维护全局和分支事务的状态，驱动全局事务提交或回滚。

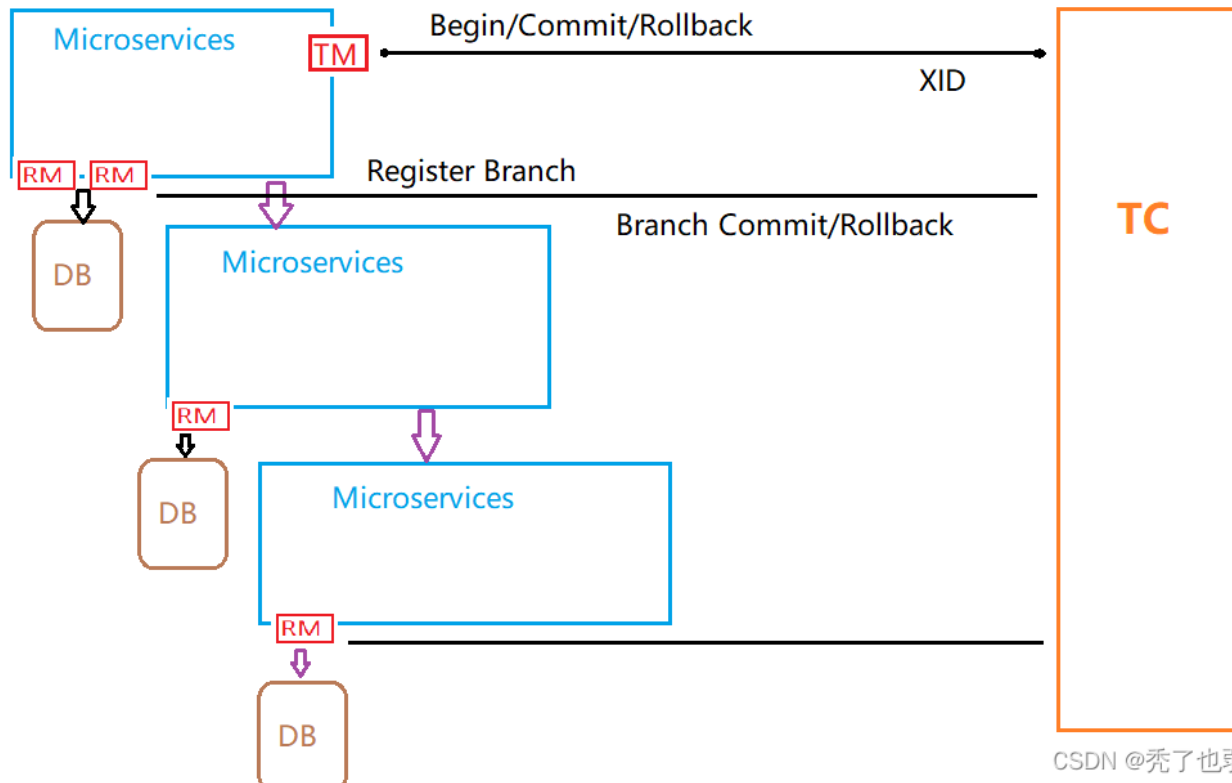
TM-事务管理者：定义全局事务的范围：开始全局事务、提交或回滚全局事务。

RM-资源管理器：管理分支事务处理的资源，与TC交谈以注册分支事务和报告分支事务的状态，并驱动分支事务提交或回滚。

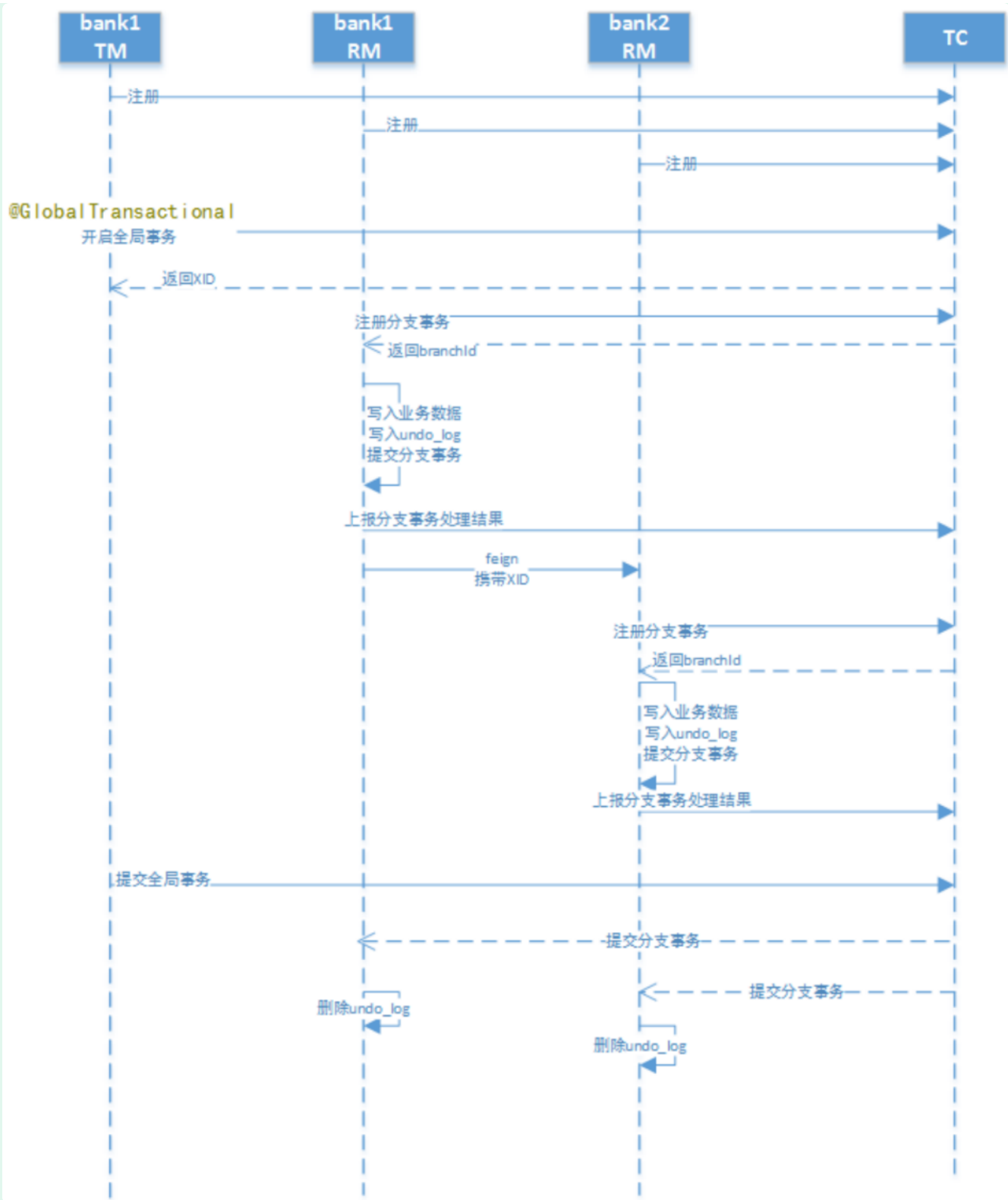
处理过程：

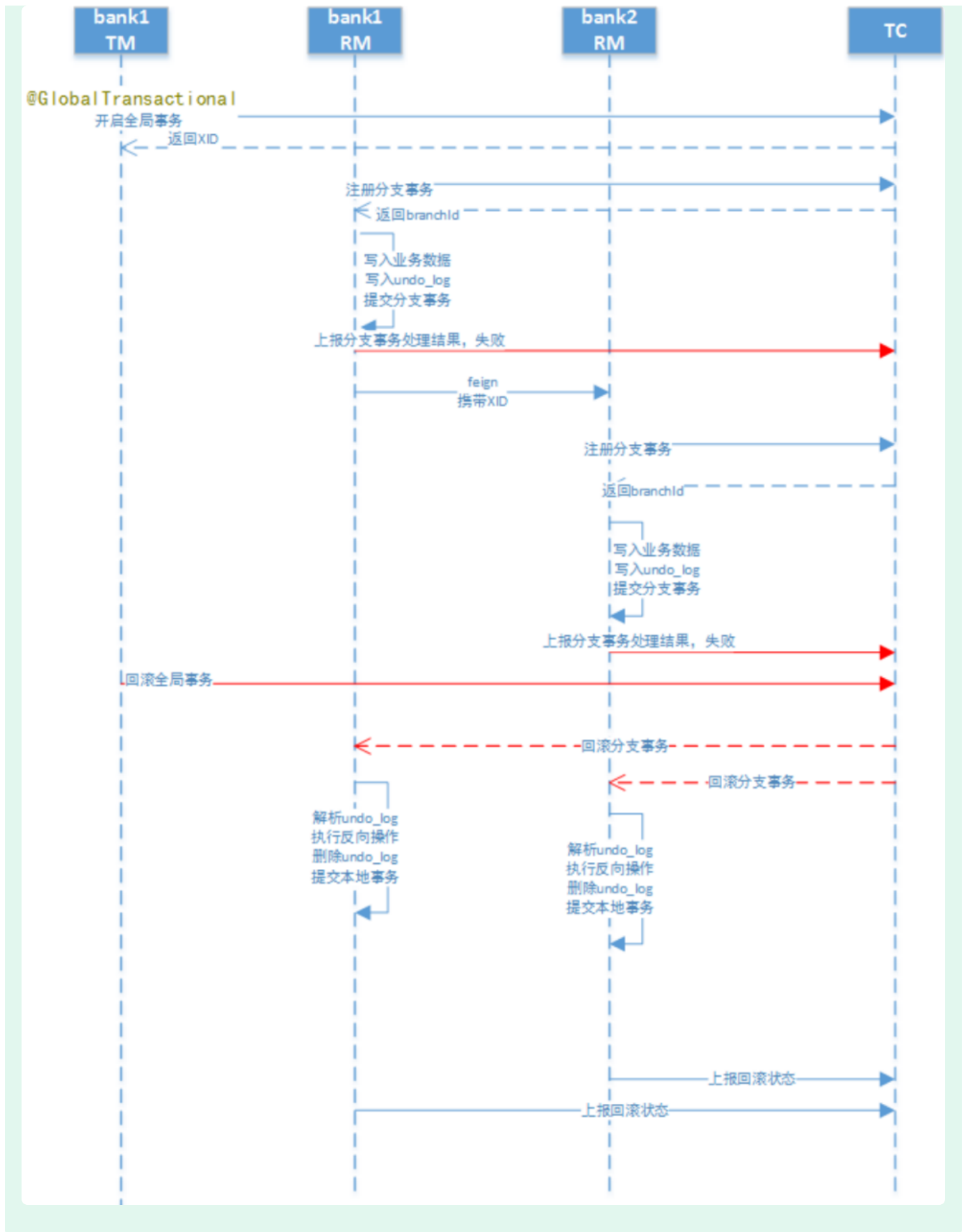
1. TM向TC申请开启一个全局事务，全局事务创建成功并生成一个全局唯一的XID。
2. XID在微服务调用链路的上下文中传播。
3. RM向TC注册分支事务，将其纳入XID对应全局事务的管辖。
4. TM向TC发起针对XID的全局提交或回滚决议。
5. TC调度XID下管辖的全部分支事务完成提交或回滚请求。

https://blog.csdn.net/A_art_xiang/article/details/123311350



CSDN @秃了也弱了。





2.4.1.4. 全业务支撑平台的分布式事务

基于数据库XA协议的DAM，类似于Seata，但DAM扩展了XADataSource数据源，支持对数据库慢sql的监控，隔离，复用数据库连接（池化）等操作。

感觉关系上是：

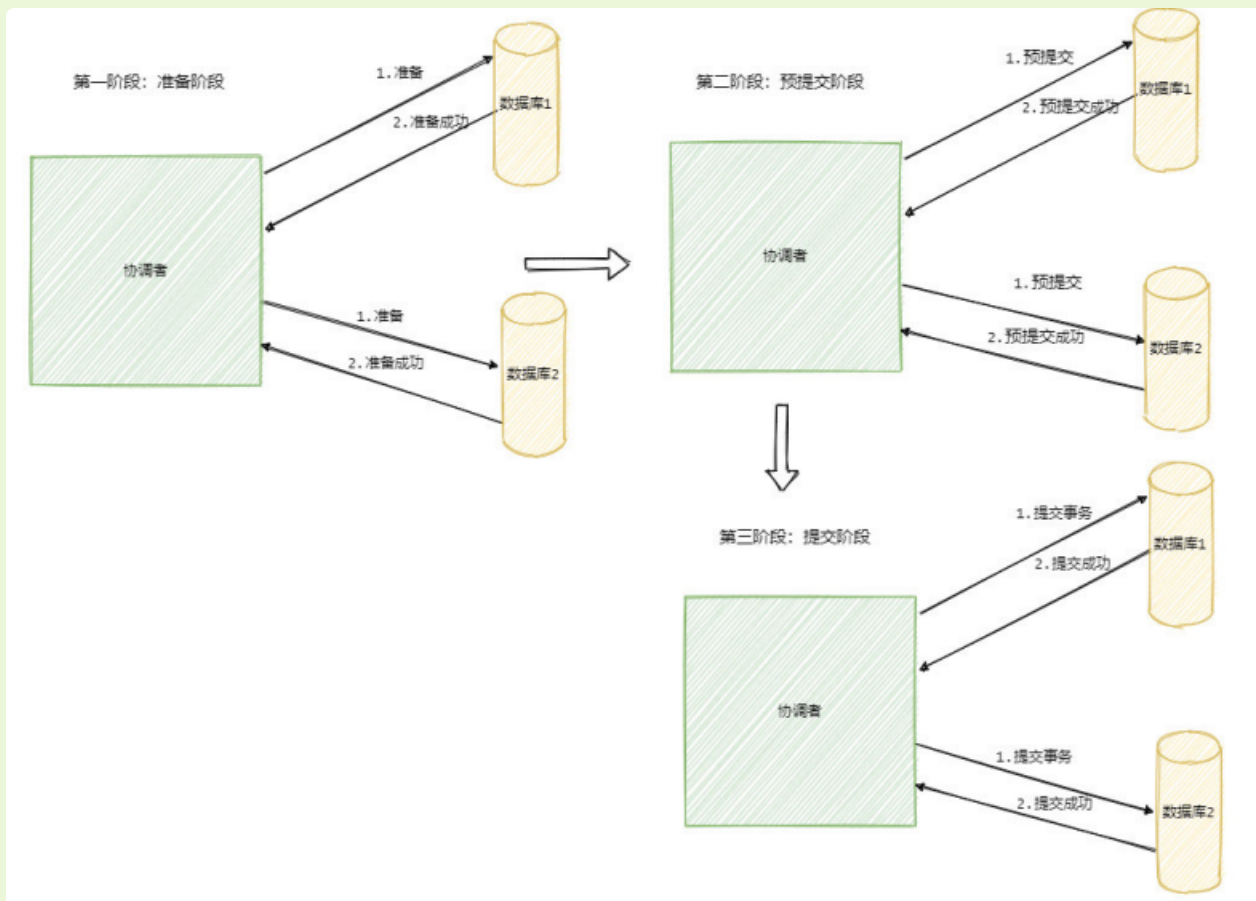
2PC -> 抽象类， XA -> 接口（类似多继承特性）， Seata && DAM (实现类)

2.4.2. 3PC，三阶段提交

3PC 相比于 2PC引入了超时机制，并且新增了一个阶段使得参与者可以利用这个阶段统一各自的状态。

包含三个阶段，准备阶段（CanCommit），预提交阶段（PreCommit），提交阶段（DoCommit）

把2PC的提交阶段变成预提交阶段和提交阶段，同时准备阶段只是询问参与者自身的状态，负载之类的，预提交阶段才是做具体的操作，但是不提交事务。



多引入一个阶段的影响：

1. 首先准备阶段的变更成不会直接执行事务，而是会先去询问此时的参与者是否有条件接这个事务，因此不会一来就干活直接锁资源，使得在某些资源不可用的情况下所有参与者都阻塞着。
2. 而预提交阶段的引入起到了一个统一状态的作用，它像一道栅栏，表明在预提交阶段前所有参与者其实还未都回应，在预处理阶段表明所有参与者都已经回应了。假如你是一位参与者，你知道自己进入了预提交状态那你就可以推断出来其他参与者也都进入了预提交状态。
3. 但是多引入一个阶段也多一个交互，因此性能会差一些，而且绝大部分的情况下资源应该都是可

用的，这样等于每次明知可用执行还得询问一次。

超时机制的影响：

1. 2PC 是同步阻塞的，协调者挂在了提交请求还未发出去的时候是最伤的，所有参与者都已经锁定资源并且阻塞等待着。
 2. 那么引入了超时机制，参与者就不会傻等了，如果是等待提交命令超时，那么参与者就会提交事务了，因为都到了这一阶段了大概率是提交的，如果是等待预提交命令超时，那该干啥就干啥了，反正本来啥也没干。
 3. 然而超时机制也会带来数据不一致的问题，比如在等待提交命令时候超时了，参与者默认执行的是提交事务操作，但是有可能执行的是回滚操作，这样一来数据就不一致了。
- 3PC 的引入是为了解决提交阶段 2PC 协调者和某参与者都挂了之后新选举的协调者不知道当前应该提交还是回滚的问题。
 - 新协调者来的时候发现有一个参与者处于预提交或者提交阶段，那么表明已经经过了所有参与者的确认
 - 了，所以此时执行的就是提交命令。
 - 所以说 3PC 就是通过引入预提交阶段来使得参与者之间的状态得到统一，也就是留了一个阶段让大家
 - 同步一下。
 - 但是这也只能让协调者知道该如果做，但不能保证这样做一定对，这其实和上面 2PC 分析一致，因为
 - 挂了的参与者到底有没有执行事务无法断定。
 - 所以说 3PC 通过预提交阶段可以减少故障恢复时候的复杂性，但是不能保证数据一致，除非挂了的那
 - 个参与者恢复。

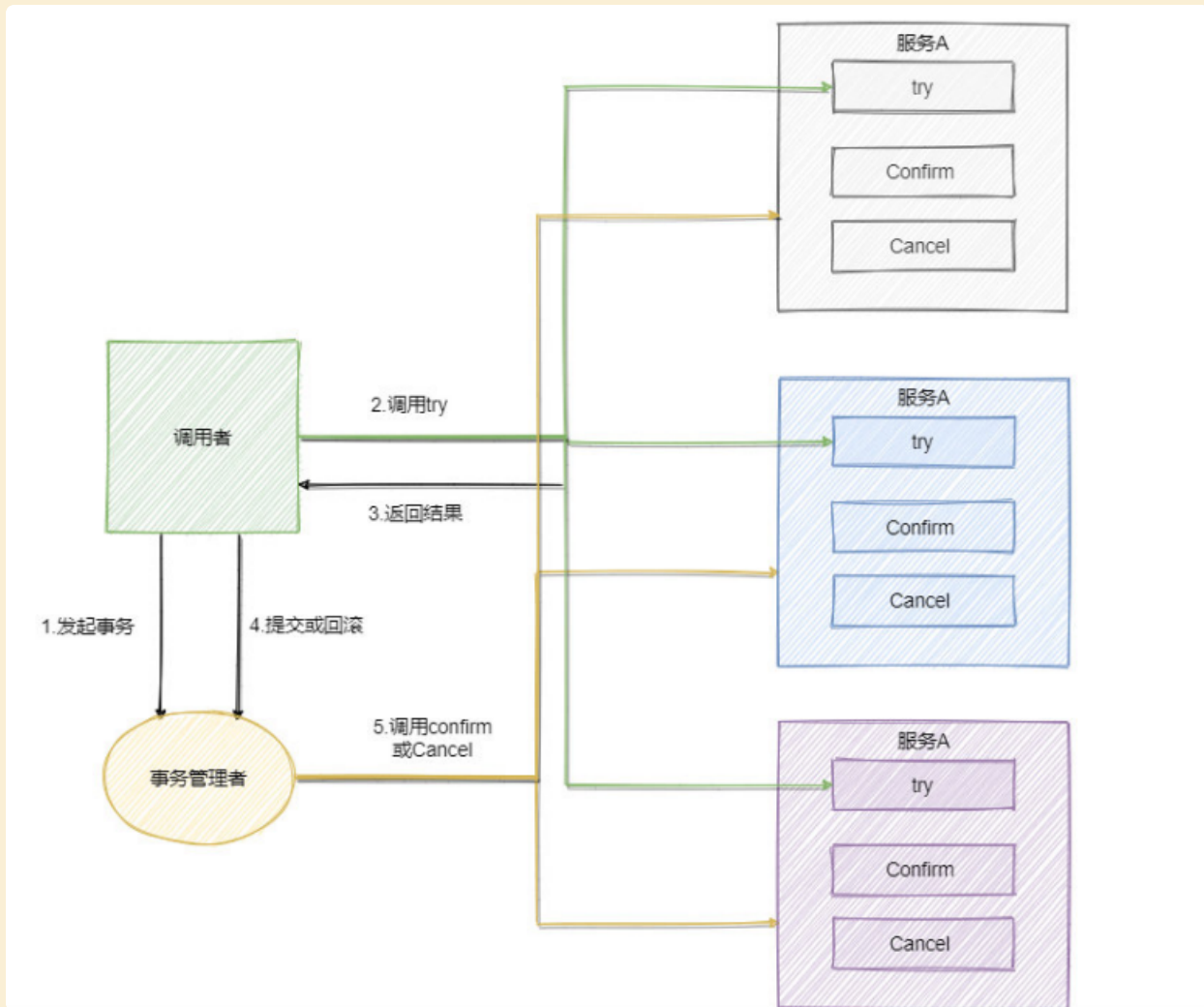
3PC 相对于 2PC 做了一定的改进：引入了参与者超时机制，并且增加了预提交阶段使得故障恢复之后协调者的决策复杂度降低，但整体的交互过程更长了，性能有所下降，并且还是会存在数据不一致问题。

所以 2PC 和 3PC 都不能保证数据 100% 一致，因此一般都需要有定时扫描补偿机制。

3PC没有找到具体的实现，只做了解。

2.4.3. TCC

TCC是Try、Confirm、Cancel，TCC要求每个分支事务实现三个操作：预处理Try、确认Confirm、撤销Cancel。Try操作做业务检查及资源预留，Confirm做业务确认操作，Cancel实现一个与Try相反的操作即回滚操作。TM首先发起所有的分支事务的try操作，任何一个分支事务的try操作执行失败，TM将会发起所有分支事务的Cancel操作，若try操作全部成功，TM将会发起所有分支事务的Confirm操作，其中Confirm/Cancel操作若执行失败，TM会进行重试。



流程还是很简单的，难点在于业务上的定义，对于每一个操作你都需要定义三个动作分别对应 Try – Confirm – Cancel 。因此 对业务的侵入较大和业务紧耦合，需要根据特定的场景和业务逻辑来设计相应的操作。

还有一点要注意，撤销和确认操作的执行可能需要重试，因此还需要保证操作的幂等。

还有因为网络等问题 try 环节没做，超时了，这时候需要执行空回滚。

还有二阶段 Cancel 接口比 Try 接口先执行，导致业务资源预留后没法继续处理的悬挂问题。

相对于 2PC、3PC，TCC 适用的范围更大，但是开发量也更大，毕竟都在业务上实现，而且有时候会发现这三个方法还真不好写。不过也因为是在业务上实现的，所以TCC可以跨数据库、跨不同的业务系统来实现事务。

2PC，3PC 都是强一致性事务

2.4.4. 本地消息表

本地消息表其实就是利用了各系统本地的事务来实现分布式事务。

本地消息表顾名思义就是会有一张存放本地消息的表，

1. 一般都是放在数据库中，然后在执行业务的时候将业务的执行和将消息放入消息表中的操作放在同一个事务中，这样就能保证消息放入本地表中业务肯定是执行成功的。
2. 然后再去调用下一个操作，如果下一个操作调用成功了好说，消息表的消息状态可以直接改成已成功。
3. 如果调用失败也没事，会有后台任务定时去读取本地消息表，筛选出还未成功的消息再调用对应的服务，服务更新成功了再变更消息的状态。
4. 这时候有可能消息对应的操作不成功，因此也需要重试，重试就得保证对应服务的方法是幂等的，而且一般重试会有最大次数，超过最大次数可以记录下报警让人工处理。

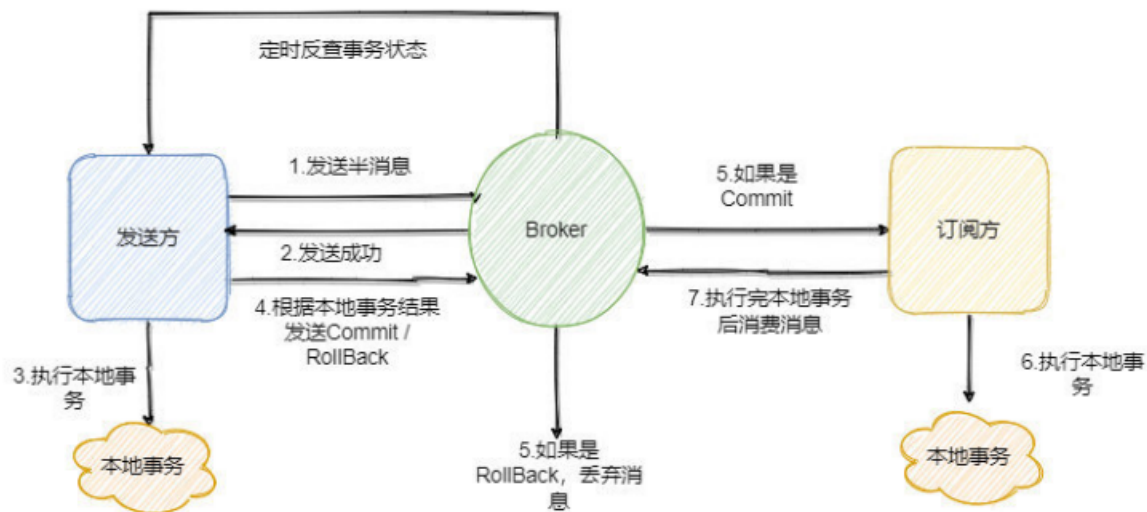
可以看到本地消息表其实实现的是最终一致性，容忍了数据暂时不一致的情况。

2.4.5. 消息事务

RocketMQ 就很好的支持了消息事务，通过消息实现事务。

1. 第一步先给 Broker 发送事务消息即半消息，半消息不是说一半消息，而是这个消息对消费者来说不可见，然后发送成功后发送方再执行本地事务。
2. 再根据本地事务的结果向 Broker 发送 Commit 或者 RollBack 命令。
3. 并且 RocketMQ 的发送方会提供一个反查事务状态接口，如果一段时间内半消息没有收到任何操作请求，那么 Broker 会通过反查接口得知发送方事务是否执行成功，然后执行 Commit 或者 RollBack 命令。
4. 如果是 Commit 那么订阅方就能收到这条消息，然后再做对应的操作，做完了之后再消费这条消息即可。
5. 如果是 RollBack 那么订阅方收不到这条消息，等于事务就没执行过。

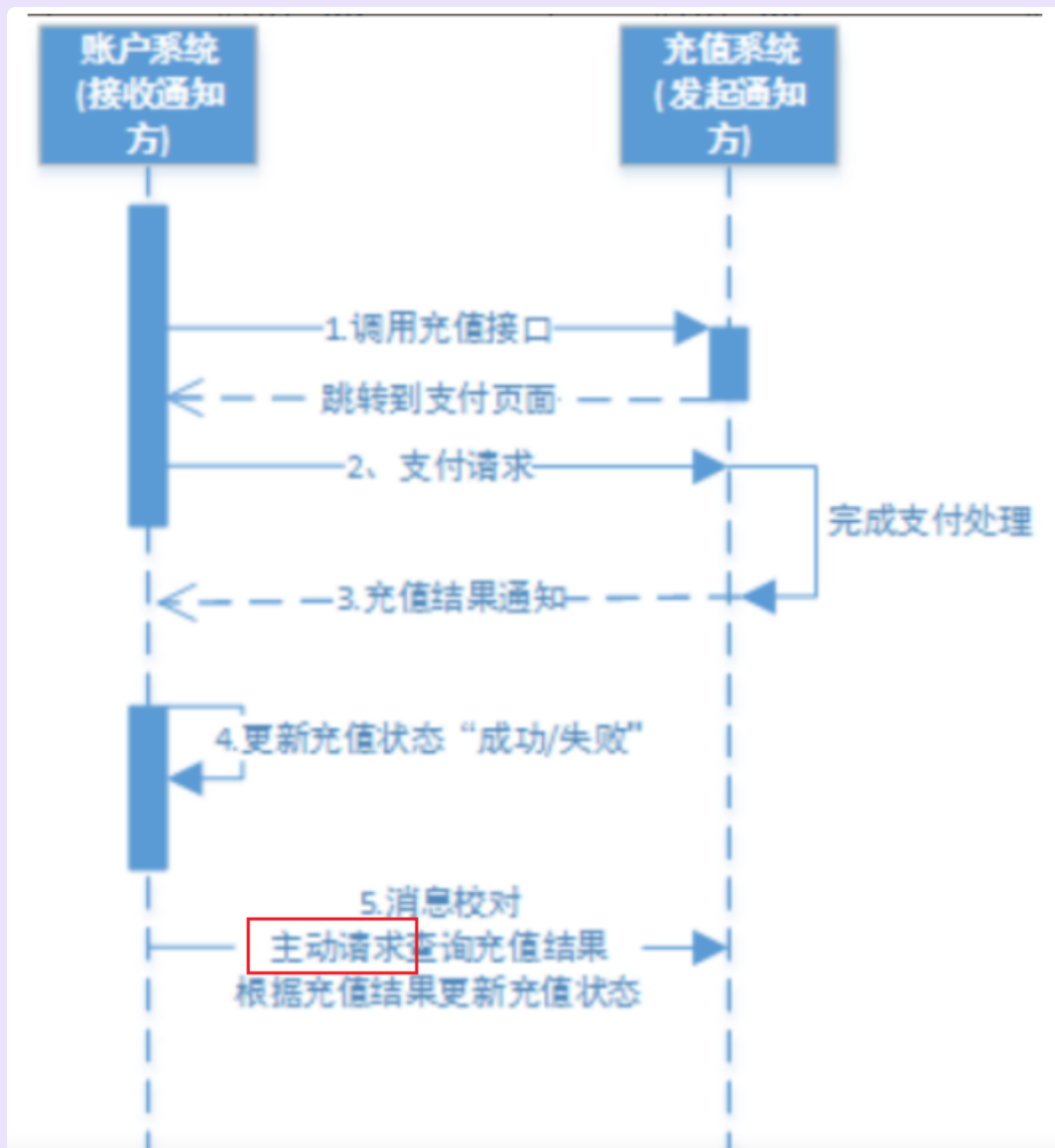
RocketMQ 提供了事务消息的功能，只需要定义好事务反查接口即可。



TCC、本地消息表、消息事务，都是实现的事务最终一致性。

本地消息表、消息事务也包括在可靠消息一致性解决方案里面。

2.4.6. 最大努力通知



一个例子如下：

交互流程：

1、账户系统调用充值系统接口

2、充值系统完成支付处理向账户系统发起充值结果通知。若通知失败，则充值系统按策略进行重复通知

3、账户系统接收到充值结果通知修改充值状态。

4、账户系统未接收到通知会主动调用充值系统的接口查询充值结果。

最大努力通知方案目标：发起通知方通过一定机制最大努力将业务处理结果通知到接收方。

具体包括：

1、有一定的消息重复通知机制。因为接收通知方可能没有接收到通知，此时要有一定的机制对消息重复通知。

2、消息校对机制。如果尽最大努力也没有通知到接收方，或者接收方消费消息后要再次消费，此时可由接收方主动向通知方查询消息信息来满足需求。

最大努力通知与可靠消息一致性有什么不同？

1、解决方案思想不同

可靠消息一致性，发起通知方需要保证将消息发出去，并且将消息发到接收通知方，消息的可靠性关键由发起通知方来保证。

最大努力通知，发起通知方尽最大的努力将业务处理结果通知为接收通知方，但是可能消息接收不到，此时需要接收通知方主动调用发起通知方的接口查询业务处理结果，通知的可靠性关键在接收通知方。

2、两者的业务应用场景不同

可靠消息一致性关注的是交易过程的事务一致，以异步的方式完成交易。

最大努力通知关注的是交易后的通知事务，即将交易结果可靠的通知出去。

3、技术解决方向不同

可靠消息一致性要解决消息从发出到接收的一致性，即消息发出并且被接收到。

最大努力通知无法保证消息从发出到接收的一致性，只提供消息接收的可靠性机制。可靠机制是，最大努力的将消息通知给接收方，当消息无法被接收方接收时，由接收方主动查询消息（业务处理结果）。

实现方式：采用MQ的ack机制实现最大努力通知

https://blog.csdn.net/A_art_xiang/article/details/127734798

	2PC	TCC	可靠消息	最大努力通知
一致性	强一致性	最终一致	最终一致	最终一致
吞吐量	低	中	高	高
实现复杂度	易	难	中	易

3. Spring事务 ---- Transactional

手动回滚事务`TransactionAspectSupport.currentTransactionStatus().setRollbackOnly();`

3.1. 事务回滚机制

Spring 事务的回滚机制指的是当事务发生异常时，如何处理已经执行的操作并回滚事务。

例如：当一个事务执行过程中发生异常时，Spring 会根据配置的事务传播机制判断当前事务是否是一个独立的事务，如果是，则会回滚整个事务，回滚操作会撤销当前事务中所有已经执行的操作，将数据恢复到事务开始前的状态。

在 Spring 中，回滚机制是通过 AOP（面向切面编程）实现的，Spring 会将事务相关的代码织入到事务切面中，在事务切面中捕获异常并执行回滚操作（默认是针对 unchecked exception 回滚）。Spring 的事务边界是在调用业务方法之前开始处理，业务方法执行完毕之后来执行 commit or rollback（Spring 默认取决于是否抛出 runtimeException）。

3.2. 事务传播级别

- **REQUIRED**: 如果当前上下文中存在事务，那么加入该事务，如果不存在事务，创建一个事务，这是默认的传播属性值。
- **SUPPORTS**: 如果当前上下文存在事务，则支持事务加入事务，如果不存在事务，则使用非事务的方式执行。
- **MANDATORY**: 如果当前上下文中存在事务，否则抛出异常。
- **REQUIRES_NEW**: 每次都会新建一个事务，并且同时将上下文中的事务挂起，执行当前新建事务完成以后，上下文事务恢复再执行。
- **NOT_SUPPORTED**: 如果当前上下文中存在事务，则挂起当前事务，然后新的方法在没有事务的环境中执行。
- **NEVER**: 如果当前上下文中存在事务，则抛出异常，否则在无事务环境上执行代码。
- **NESTED**: 如果当前存在一个事务，那么它会在当前事务内部创建一个嵌套事务。这个嵌套事务可以独立于外部事务进行提交或回滚，只回滚内层事务。如果当前没有事务，它会行为像 **PROPAGATION_REQUIRED** 一样，创建一个新事务。

事务挂起：

1. 当需要开启一个新事务时，如果当前已经存在一个活动事务，Spring会首先将这个活动事务挂起，即保存其状态（例如，通过数据库连接等相关资源）。
2. 然后，Spring会创建一个新的事务，这个新事务将独立于之前挂起的事务运行。
3. 在新事务执行完毕后（提交或回滚），Spring会恢复之前挂起的事务，并允许其继续执行。

3.3. 注解底层实现

@Transactional注解的底层实现是通过AOP（面向切面编程）来实现的。在Spring框架中，使用了TransactionInterceptor拦截器来实现@Transactional注解的功能。TransactionInterceptor是一个AOP拦截器，它在目标方法执行之前和之后分别开启和提交事务。

具体来说，当方法上标记了@Transactional注解时，Spring会在运行时生成一个代理对象，该代理对象会拦截对该方法的调用，并在执行方法之前开启一个事务，然后在执行完成后根据方法的执行结果来决定是提交事务还是回滚事务。

在底层实现中，TransactionInterceptor使用了TransactionManager来管理事务，而TransactionManager则使用了底层的数据源来实现事务的管理。在事务提交或回滚时，TransactionInterceptor会根据事务的状态来决定是否提交或回滚，同时还会负责管理事务的传播行为和隔离级别等事务属性。

<https://blog.csdn.net/zhoqua697/article/details/130372920>

3.4. 事务失效场景

- 数据库不支持事务：Spring事务是依靠数据库的，如Mysql的MysAm引擎不支持事务。
- 方法没有被public修饰：底层做了判断，在AbstractFallbackTransactionAttributeSource类的computeTransactionAttribute方法中有一个判断，如果目标方法不是public，则TransactionAttribute返回null，不支持事务。
- 方法内部调用：如在非事务的方法里面调用事务方法，事务也会失效
- 方法被final，static修饰：spring事务底层使用了AOP，需要在方法对应位置添加实现，但是final方法无法被重写，static无法生成动态代理。
- 服务未被Spring容器管理
- 多线程调用：多个线程拥有多个不同的数据库连接，不能同时提交和回滚。
- 事务传播级别错误：使用了NOT_SUPPORTED、NEVER等事务传播级别
- spring工程未配置事务管理器，未开启事务
- 自己吞了异常、手动抛错了异常、在catch中出现自定义异常之外的异常：catch了异常Exception，但spring事务是在runtime Exception才能回滚。这种情况可以设置参数rollbackFor = Exception.class

<https://blog.csdn.net/wwg1234wwg1234/article/details/123322545>

4. 分布式ID

5. 分布式session

6. 常见例子

6.1. 例子一

假定有三个服务(订单服务、账户服务、库存服务), 现有个订单支付的需求, 支付提交之后同时需要更改订单状态, 需要扣减账户余额, 需要扣减库存。

问:

(1)如何解决强一致性问题,你能想到哪些方案?你觉得哪种方案最好?说说理由?

分布式事务, 2PC, 3PC, seata框架

(2)如果允许最终一致性, 你会怎么处理? 如何提高开发效率?

消息队列。

(3)高并发情况下, 程序中你如何确保账户和库存均不会被超扣, 怎么提高执行效率?

乐观锁或者是分布式锁

(1)如何解决强一致性问题：

在解决强一致性问题时，通常我们需要保证所有的服务操作要么全部成功，要么全部失败。这可以通过分布式事务来实现。以下是一些可能的方案：

两阶段提交（2PC）或三阶段提交（3PC）：这是经典的分布式事务解决方案，通过准备阶段和提交阶段确保所有服务的数据一致性。但是，这种方式存在阻塞、单点故障等问题，性能也相对较低。

分布式事务框架：如Seata（Simple Extensible Autonomous Transaction Architecture），它提供了全局事务管理、分支事务管理等功能，可以简化分布式事务的编程工作。

本地消息表：订单服务在本地数据库中维护一个消息表，记录需要通知其他服务进行操作的消息。通过定时任务或轮询的方式，检查消息表并调用其他服务的接口进行更新。

至于哪种方案最好，这取决于具体的业务场景和性能要求。如果业务对性能要求较高，且能够接受短暂的数据不一致，那么本地消息表可能是一个较好的选择。如果业务对数据一致性要求极高，且可以接受一定的性能损耗，那么可以考虑使用分布式事务框架。

(2)如果允许最终一致性：

如果允许最终一致性，即允许在一定时间内存在数据不一致的情况，那么可以采用补偿事务或基于事件的异步通知机制。

补偿事务：当某个服务的操作失败时，通过另一个操作来补偿之前失败的操作，使得最终数据达到一致状态。

基于事件的异步通知：当订单支付成功后，发布一个事件，账户服务和库存服务订阅这个事件，并异步处理。这种方式能够降低系统的耦合度，提高系统的可扩展性。

为了提高开发效率，可以采用消息队列（如Kafka、RabbitMQ等）作为事件的发布和订阅平台，通过订阅相应的事件来处理业务逻辑。此外，还可以使用事件溯源（Event Sourcing）或CQRS（Command Query Responsibility Segregation）等架构模式来简化分布式系统的开发。

(3)高并发情况下如何确保账户和库存不被超扣，以及如何提高执行效率：

在高并发情况下，为了避免账户和库存被超扣，可以采取以下策略：

分布式锁：使用Redis等分布式锁工具，确保同一时间只有一个请求能够操作账户或库存。

乐观锁：在数据库层面使用乐观锁机制，通过版本号或时间戳等方式判断数据是否被其他事务修改过。

预扣减：在支付前先进行账户和库存的预扣减，如果预扣减成功再进行支付操作。

为了提高执行效率，可以考虑以下措施：

缓存：对账户余额和库存数量等热点数据进行缓存，减少数据库访问次数。

批量处理：将多个请求合并成一批进行处理，减少数据库交互次数。

异步化：将非关键路径的操作异步化，如发送通知、记录日志等，避免阻塞主线程。

此外，还可以通过水平扩展、负载均衡等技术手段来提高系统的整体处理能力和执行效率。

调优

1. 如何监控和调优java应用程序性能

A. 性能监控

1. 使用性能监控工具：

- JConsole和VisualVM：这两个是Java自带的性能监控工具，可以实时监控Java应用程序的内存使用、线程活动、类加载等情况。它们还可以进行堆转储和线程转储，有助于定位性能问题。
- ****Java Mission Control (JMC)****：这是一个功能强大的Java性能分析工具，可以监控、分析和管理工作性能。它提供了丰富的性能指标和可视化工具，帮助开发人员深入了解应用程序的性能瓶颈。
- JProfiler和YourKit：这些是第三方的性能监控工具，提供了丰富的性能监控选项，包括CPU、内存、线程和数据库操作的监控。
- ****GCViewer和MAT (Memory Analyzer Tool)****：这些工具可以帮助分析垃圾收集的性能和内存使用情况。
- 日志分析：通过分析应用程序的日志文件，可以发现一些性能问题和错误。例如，通过搜索特定的错误信息或异常信息，可以找到潜在的代码问题和配置错误。

2. 压力测试：

通过模拟实际用户操作，对应用程序进行压力测试，可以发现性能瓶颈和错误。常用的压力测试工具包括JMeter和Gatling等。

B. 性能调优

1. JVM调优：

- 调整JVM参数：根据程序的需求调整JVM的堆大小、栈大小等参数，以提高程序的运行效率。
- 使用JIT编译器：JIT编译器可以将Java字节码转换为本地机器码，提高程序的执行速度。
- 进行GC调优：根据程序的内存使用特点选择合适的垃圾回收器，并调整其参数以优化垃圾回收的性能。

2. 代码优化：

减少循环次数、避免重复计算、使用更高效的算法和数据结构等。

内存管理：优化内存管理可以包括减少内存使用、及时释放不再使用的对象、合理配置垃圾回收参数等。

3. 并发和多线程：

合理地使用并发和多线程可以提高程序的并发处理能力和响应性能。同时，需要避免不正确的并发处理导致的性能问题，如死锁和竞争条件。

请注意，性能监控和调优是一个持续的过程，需要根据应用程序的实际情况和需求进行定期的检查和调整。此外，在进行性能调优时，建议先在测试环境中验证更改的效果，以确保不会对生产环境造成不良影响。

服务器 && Linux

nginx

1. nginx 配置文件的结构是什么

Nginx配置文件的结构主要包括以下几个部分：

全局块：定义全局性的配置参数，影响整个Nginx服务的运行。例如，可以指定运行Nginx的用户和用户组（`user nginx;`），指定工作进程数量（`worker_processes auto;`），定义错误日志文件路径（`error_log /var/log/nginx/error.log;`）等。

事件块：配置Nginx服务器处理连接的工作方式。例如，指定最大连接数（`worker_connections 1024;`）。

HTTP块：包含与HTTP服务相关的配置，包括服务器、位置和其他HTTP相关指令。可以加载mime.types文件（`include /etc/nginx/mime.types;`），定义访问日志格式和路径等。

服务器块：定义虚拟主机的配置，可以包含多个。每个服务器块内可以配置监听端口、域名、根目录等。

位置块：定义请求的处理方式，通常位于服务器块内。位置块可以根据不同的URI匹配模式执行不同的操作，例如代理请求到后端服务器、重定向等。

配置文件通常位于/etc/nginx/目录下，主配置文件名为nginx.conf。在配置文件中，每条指令由指令名称和参数组成，并且每条指令都有自己的语法格式和作用。

请注意，Nginx的配置文件具有模块化结构，并且可以通过include指令包含其他配置文件，从而实现配置的模块化管理和复用。

此外，Nginx还支持反向代理和负载均衡功能，这些功能可以通过在配置文件中设置相应的指令来实现。例如，反向代理可以通过proxy_pass指令来实现，而负载均衡则可以通过在upstream块中定义后端服务器列表，并在server块中通过proxy_pass指令将请求转发到后端服务器来实现。

详解：https://blog.csdn.net/m0_62231324/article/details/132524647

2. 如何优化nginx性能

- 启用Gzip压缩：通过gzip压缩技术优化传输效率，能够显著地加快页面的访问速度，提升用户的体验感和网站的质量。在Nginx配置文件中启用gzip压缩功能，可以指定要压缩的响应内容类型、压缩级别以及开启或关闭gzip功能等。
- 调整缓存设置：Nginx内置了缓存功能，可以通过配置缓存指令来缓存静态文件和动态内容。合理地配置缓存，可以在用户再次访问时直接从缓存中读取，减少对后端服务器的请求，提高响应速度。
- 调整并发连接数：根据实际情况调整Nginx的并发连接数限制，以支持更多的并发请求。可以通过修改worker_processes参数进行调整。
- 优化事件驱动模型：Nginx使用事件驱动的架构，可以高效地处理并发连接。通过优化事件驱动模型来提高性能，例如调整事件驱动模型的参数、使用更高效的事件驱动库等。
- 启用keepalive连接：通过启用keepalive连接，可以在单个TCP连接上处理多个HTTP请求，减少了连接建立的开销，提高了性能。
- 优化TCP参数：在操作系统层面，可以通过调整TCP参数来优化网络性能。例如，调整TCP的初始拥塞窗口大小、初始重传超时时间等。
- 调整worker_processes参数：worker_processes参数定义了Nginx可以同时处理多少个工作进程。根据服务器的CPU核心数和负载情况，可以适当调整该参数以提高Nginx的性能。
- 启用零拷贝技术：零拷贝技术可以减少数据在内存中的拷贝次数，从而提高性能。在Nginx中，可以通过配置sendfile指令来启用零拷贝技术。
- 合并小文件：对于大量小文件的访问，可以通过合并小文件为单个文件，减少对文件的访问次数，从而提高性能。

请注意，在进行任何优化之前，都应该先备份现有的Nginx配置文件，并测试新配置对性能和稳定性的影响。同时，根据实际的业务需求和服务器环境，选择合适的优化策略并进行调整。

Linux

1. linux的防火墙的两个主要工具是什么

Linux的防火墙的两个主要工具是iptables和firewalld。

iptables是Linux上最传统的防火墙工具，工作在内核层面，提供了强大的包过滤功能。iptables基于Netfilter框架，允许管理员定义规则来控制进出系统的网络流量。

而firewalld是一种动态管理防火墙的工具，使用区域和服务的概念来管理网络流量，它主要用于需要频繁更改网络设置的环境，如服务器和复杂的网络结构。

在选择使用哪个工具时，需要考虑具体需求、所在环境的复杂性以及对Linux防火墙的熟悉程度。iptables更适合需要精细控制网络流量的高级用户和系统管理员，而firewalld则提供了更加直观的区域和服务管理方法，适用于需要动态防火墙管理的环境。

2. 分别说明linux的top, scp, yum, crontab, rsync, mount, traceroute, netstat的作用

1. top

top 是一个动态显示系统中各个进程的资源占用状况的监视工具。它可以实时地显示系统中各个进程的资源占用状况，类似于 Windows 的任务管理器。通过 top，用户可以查看 CPU 使用率、内存使用率、正在运行的进程等信息。

2. scp

scp 是 secure copy 的缩写，用于在本地主机和远程主机之间复制文件。它是基于 SSH 协议进行安全传输的。使用 scp，你可以将本地文件复制到远程主机，或者从远程主机复制文件到本地。

3. yum

yum 是 Yellowdog Updater Modified 的缩写，是 Fedora、Red Hat Enterprise Linux 和 CentOS 等 Linux 发行版的软件包管理器。它允许用户从指定的软件仓库中自动下载、安装、升级和删除软件包及其依赖。

4. crontab

crontab 是用于设置周期性被执行的任务的工具。通过编辑 crontab 文件，用户可以定义在特定时间（如每天、每周或每月的某个时间）自动执行的任务或命令。这对于需要定期备份、监控或其他自动化任务非常有用。

5. rsync

rsync 是一个远程文件同步工具，用于快速、增量地复制文件和目录到本地或远程主机。它只复制源和目标之间的差异部分，因此比简单的复制命令更加高效。rsync 常用于备份和镜像。

6. mount

mount 命令用于挂载文件系统。在 Linux 中，你可以将存储设备（如硬盘、USB 驱动器、CD/DVD 等）上的文件系统挂载到目录树中的某个位置，以便访问其中的文件和目录。mount 命令用于执行这一操作。

7. traceroute

traceroute 是一个显示数据包从源主机到目标主机之间经过的路由路径的工具。它发送一系列的数据包到目标主机，并记录每个数据包经过的路由节点，从而帮助用户了解网络路径和可能的网络问题。

8. netstat

netstat 是 "network statistics" 的缩写，用于显示网络连接、路由表、接口统计等网络相关信息。通过 netstat，用户可以查看当前系统的网络连接状态、监听的端口、网络接口的统计信息等，有助于诊断网络问题。

